Relational Database: Definitions

- *Relational database:* a set of *relations*.
- *Relation:* made up of 2 parts:
  - *Instance* : a *table*, with rows and columns. #rows = *cardinality*, #fields = *degree / arity*
  - *Schema* : specifies name of relation, plus name and type of each column.
    - E.g. Students(*sid*: string, *name*: string, *login*: string, *age*: integer, *gpa*: real)
- Can think of a relation as a *set* of rows or *tuples*. (i.e., all rows are distinct)

Example Instance of Students Relation

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

- Cardinality = 3, degree = 5 , all rows distinct

Relational Query Languages

- A major strength of the relational model: supports simple, powerful *querying* of data.
- Queries can be written intuitively, and the DBMS is responsible for efficient evaluation.
  - The key: precise semantics for relational queries.
  - Allows the optimizer to extensively re-order operations, and still ensure that the answer does not change.

```
SELECT  S.name, E.cid, E.grade
FROM  Students S, Enrolled E
WHERE  S.sid=E.sid AND S.age >18 AND E.ID='ITCS6160'
```

The SQL Query Language

- Developed by IBM (system R) in the 1970s
- Need for a standard since it is used by many vendors
- Standards:
  - SQL-86
  - SQL-89 (minor revision)
  - SQL-92 (major revision, current standard)
  - SQL-99 (major extensions)

Creating Relations in SQL

- Creates the Students relation. Observe that the type (domain) of each field is specified, and enforced by the DBMS whenever tuples are added or modified.
- As another example, the Enrolled table holds information about courses that students take.

```
CREATE TABLE Students
        (sid CHAR(20),
         name CHAR(20),
         login CHAR(10),
         age INTEGER,
         gpa REAL)
```

```
CREATE TABLE Enrolled
        (sid CHAR(20),
         cid CHAR(20),
         grade CHAR(2))
```

Destroying and Altering Relations

DROP TABLE Students

- Destroys the relation Students. The schema information *and* the tuples are deleted.

ALTER TABLE Students
ADD COLUMN firstYear: integer

- The schema of Students is altered by adding a new field; every tuple in the current instance is extended with a *null* value in the new field.

Adding and Deleting Tuples

- Can insert a single tuple using:

```
INSERT INTO  Students (sid, name, login, age, gpa)
VALUES  (53699, 'Green ', 'green@ee', 18, 3.5)
```

- More inserts:

```
INSERT INTO  Students (sid, name, login, age, gpa)
VALUES  (53666, 'Jones', 'jones@cs', 18, 3.4)
INSERT INTO  Students (sid, name, login, age, gpa)
VALUES  (53688, 'Smith ', 'smith@eecs', 18, 3.2)
INSERT INTO  Students (sid, name, login, age, gpa)
VALUES  (53650, 'Smith ', 'smith@math', 19, 3.8)
```

Adding and Deleting Tuples (continued)

- Students relation after inserts:

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8
53600	Green	green@ee	18	3.5

Adding and Deleting Tuples (continued)

- Can delete all tuples satisfying some condition (e.g., name = Smith):

```
DELETE
FROM Students S
WHERE S.name = 'Smith'
```

- Students instance after delete:

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53600	Green	green@ee	18	3.5

The SQL Query Language

- To find all 18 year old students, we can write:

```
SELECT  *
FROM  Students S
WHERE  S.age=18
```

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@ee	18	3.2

- To find just names and logins, replace the first line:

```
SELECT  S.name, S.login from Students S
```

Adding and Deleting Tuples (continued)

- Insert tuples into the Enrolled instance:

```
INSERT INTO  Enrolled (sid, cid, grade)
VALUES  ('53831', 'Carnatic 101', 'C')
INSERT INTO  Enrolled (sid, cid, grade)
VALUES  ('53831', 'Reggae 203', 'B')
INSERT INTO  Enrolled (sid, cid, grade)
VALUES  ('53650', 'Topology 112', 'A')
INSERT INTO  Enrolled (sid, cid, grade)
VALUES  ('53666', 'History 105', 'B')
```

Querying Multiple Relations

- What does the following query compute?

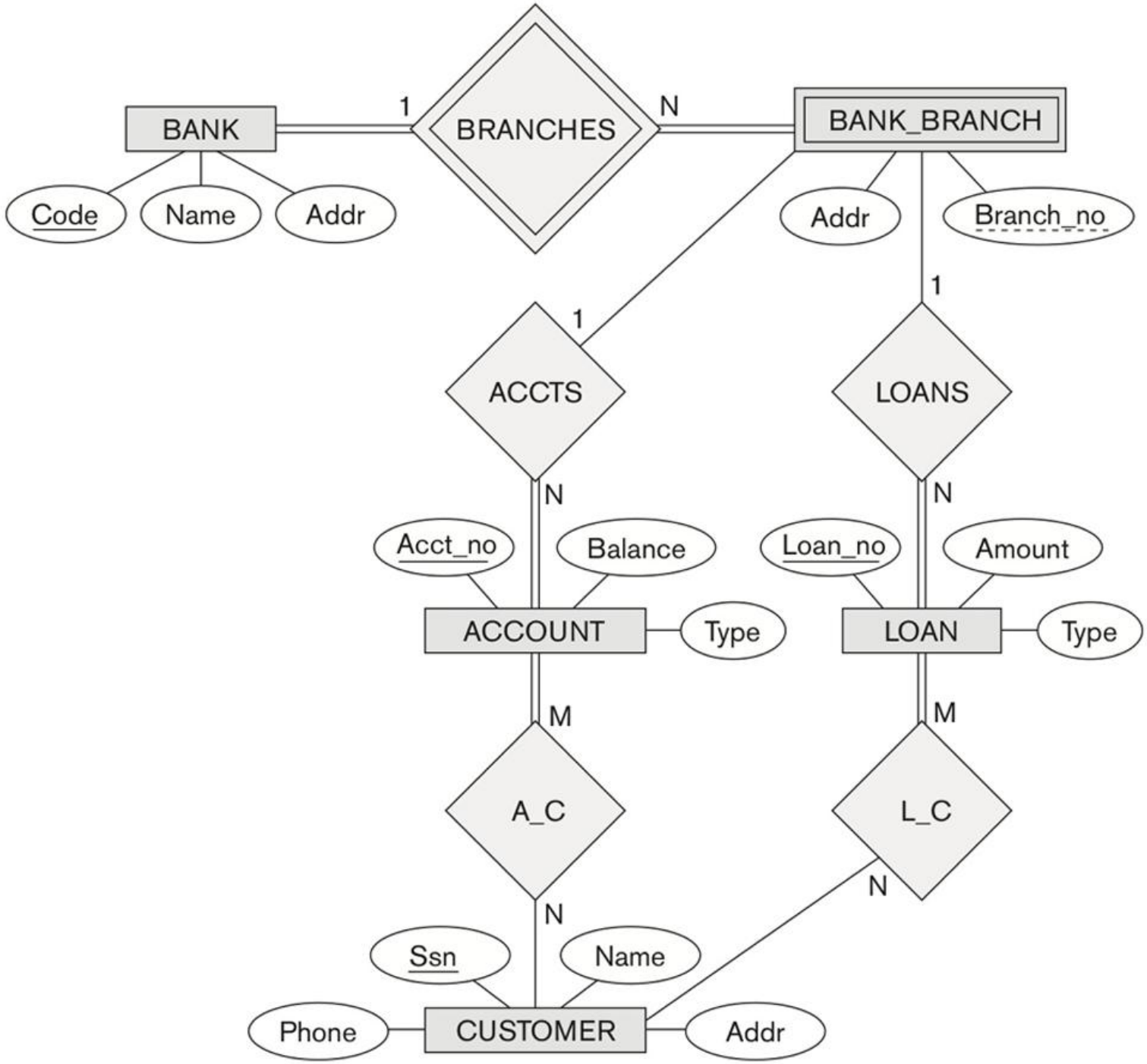
```
SELECT  S.name, E.cid
FROM  Students S, Enrolled E
WHERE  S.sid=E.sid AND E.grade='B'
```

Given the following instance of Enrolled :

sid	cid	grade
53831	Carnatic101	C
53831	Reggae203	B
53650	Topology112	A
53666	History105	B

we get:

S.name	E.cid
Jones	History 105


BANK
BRANCHES
BANK_BRANCH
1
N
Code
Name
Addr
Addr
Branch_no
1
1
ACCTS
LOANS
N
N
Acct_no
Balance
Loan_no
Amount
ACCOUNT
Type
LOAN
Type
M
M
A_C
L_C
N
N
Ssn
Name
Phone
CUSTOMER
Addr

1. List the strong (nonweak) entity types in the ER diagram.

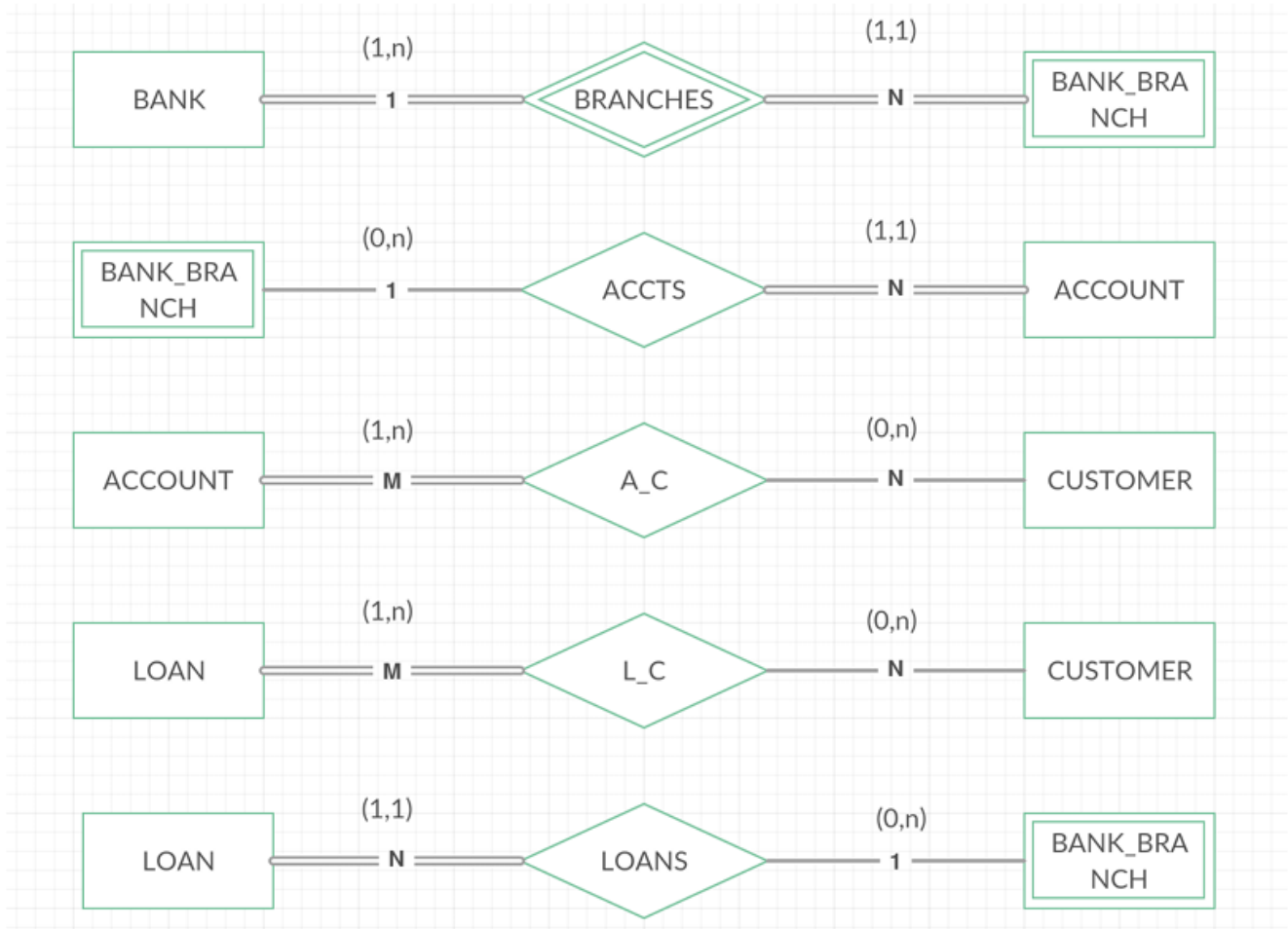
2. Is there a weak entity type? If so, give its name, partial key, and identifying relationship.

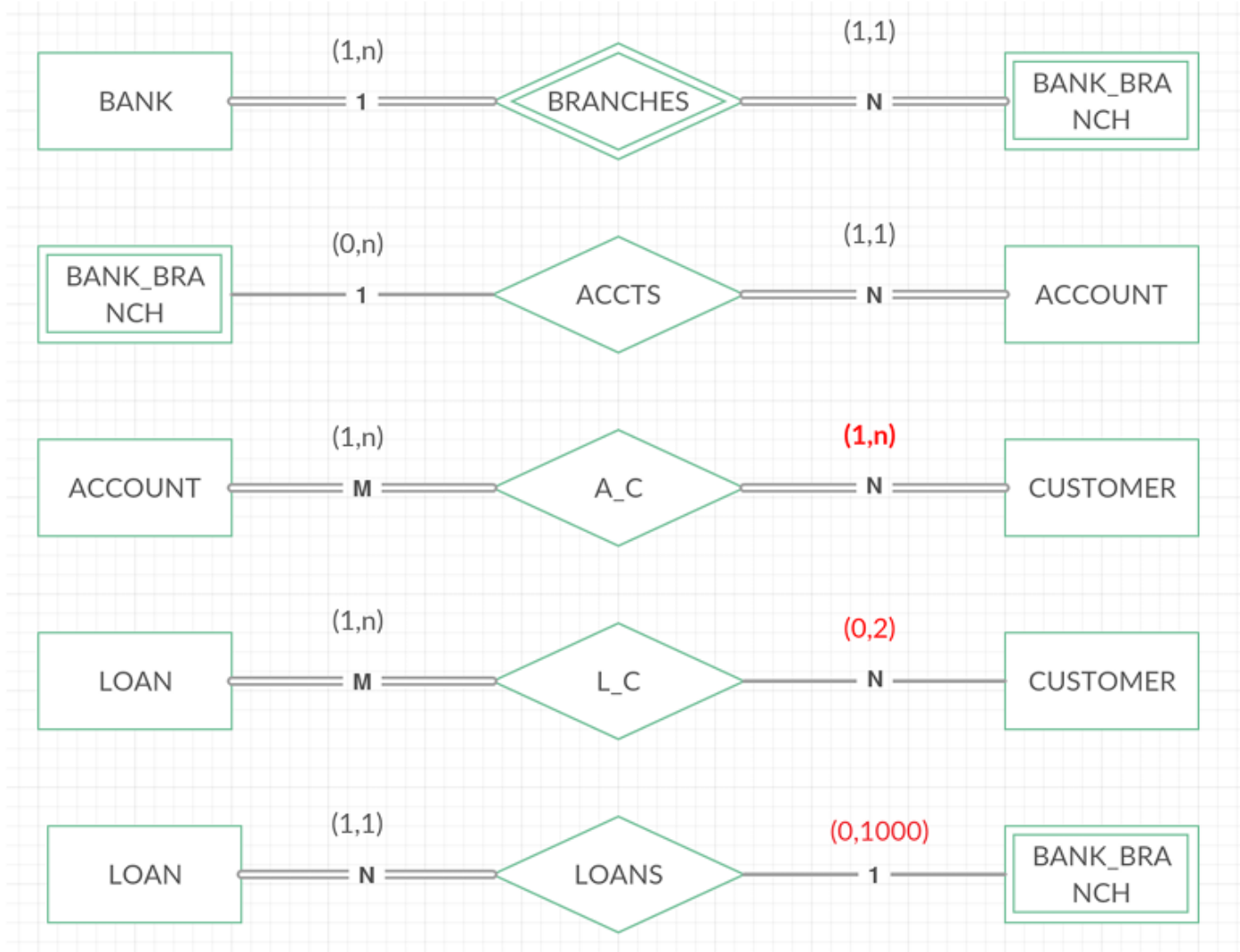
3. What constraints do the partial key and the identifying relationship of the weak entity type specify in this diagram?

4. List the names of all relationship types, and specify the (min, max) constraint on each participation of an entity type in a relationship type.

5. List concisely the user requirements that led to this ER schema design.

6. Suppose that every customer must have at least one account but is restricted to at most two loans at a time, and that a bank branch cannot have more than 1,000 loans. How does this show up on the (min, max) constraints?


BANK
(1,n)
1
BRANCHES
(1,1)
N
BANK_BRANCH
BANK_BRANCH
(0,n)
1
ACCTS
(1,1)
N
ACCOUNT
ACCOUNT
(1,n)
M
A_C
(0,n)
N
CUSTOMER
LOAN
(1,n)
M
L_C
(0,n)
N
CUSTOMER
LOAN
(1,1)
N
LOANS
(0,n)
1
BANK_BRANCH


BANK
(1,n)
1
BRANCHES
(1,1)
N
BANK_BRANCH
BANK_BRANCH
(0,n)
1
ACCTS
(1,1)
N
ACCOUNT
ACCOUNT
(1,n)
M
A_C
(1,n)
N
CUSTOMER
LOAN
(1,n)
M
L_C
(0,2)
N
CUSTOMER
LOAN
(1,1)
N
LOANS
(0,1000)
1
BANK_BRANCH

Draw ERD Crow’s foot notation based on following business rules.

A salesperson may manage many other salespeople. A salesperson is managed by only one salespeople. A salesperson can be an agent for many customers. A customer is managed by one salespeople. A customer can place many orders. An order can be placed by one customer. An order lists many inventory items. An inventory item may be listed on many orders. An inventory item is assembled from many parts. A part may be assembled into many inventory items. Many employees assemble an inventory item from many parts. A supplier supplies many parts. A part may be supplied by many suppliers.

Step 1: Identify the Entities

- SALESPERSON: Represents sales staff members
- CUSTOMER: Represents clients who place orders
- ORDER: Represents sales transactions
- INVENTORY_ITEM: Represents products available for sale
- PART: Represents components used to assemble inventory items
- SUPPLIER: Represents companies that provide parts
- EMPLOYEE: Represents workers who assemble items

Step 2: Add Primary Keys For each entity, add a unique identifier:

- SALESPERSON: salesperson_id (PK)
- CUSTOMER: customer_id (PK)
- ORDER: order_id (PK)
- INVENTORY_ITEM: item_id (PK)
- PART: part_id (PK)
- SUPPLIER: supplier_id (PK)
- EMPLOYEE: employee_id (PK)

Step 3: Establish Relationships and Foreign Keys

SALESPERSON Self-Relationship:

- Add manager_id (FK) to SALESPERSON
- One salesperson manages many salespeople
- Each salesperson has one manager
- This creates a hierarchical structure

SALESPERSON-CUSTOMER:

- Add salesperson_id (FK) to CUSTOMER
- One salesperson manages many customers
- Each customer has exactly one salesperson

CUSTOMER-ORDER:

- Add customer_id (FK) to ORDER
- One customer can place many orders
- Each order belongs to one customer

ORDER-INVENTORY_ITEM:

- Create a junction table ORDER_ITEMS with:
  - order_id (FK)
  - item_id (FK)
- Many orders can include many inventory items

INVENTORY_ITEM-PART:

- Create a junction table ITEM_PARTS with:
  - item_id (FK)
  - part_id (FK)
- Many items can be assembled from many parts

SUPPLIER-PART:

- Create a junction table SUPPLIER_PARTS with:
  - supplier_id (FK)
  - part_id (FK)
- Many suppliers can supply many parts

EMPLOYEE-INVENTORY_ITEM:

- Create a junction table ASSEMBLY_WORKERS with:
  - employee_id (FK)
  - item_id (FK)
- Many employees can assemble many items

Step 4: Add Essential Attributes For each entity, add relevant fields:

SALESPERSON:
- name
- contact_info

CUSTOMER:
- name
- contact_info

ORDER:
- order_date
- total_amount

INVENTORY_ITEM:
- name
- description
- price

PART:
- name
- description
- cost

SUPPLIER:
- name
- contact_info

EMPLOYEE:
- name
- department
- contact_info

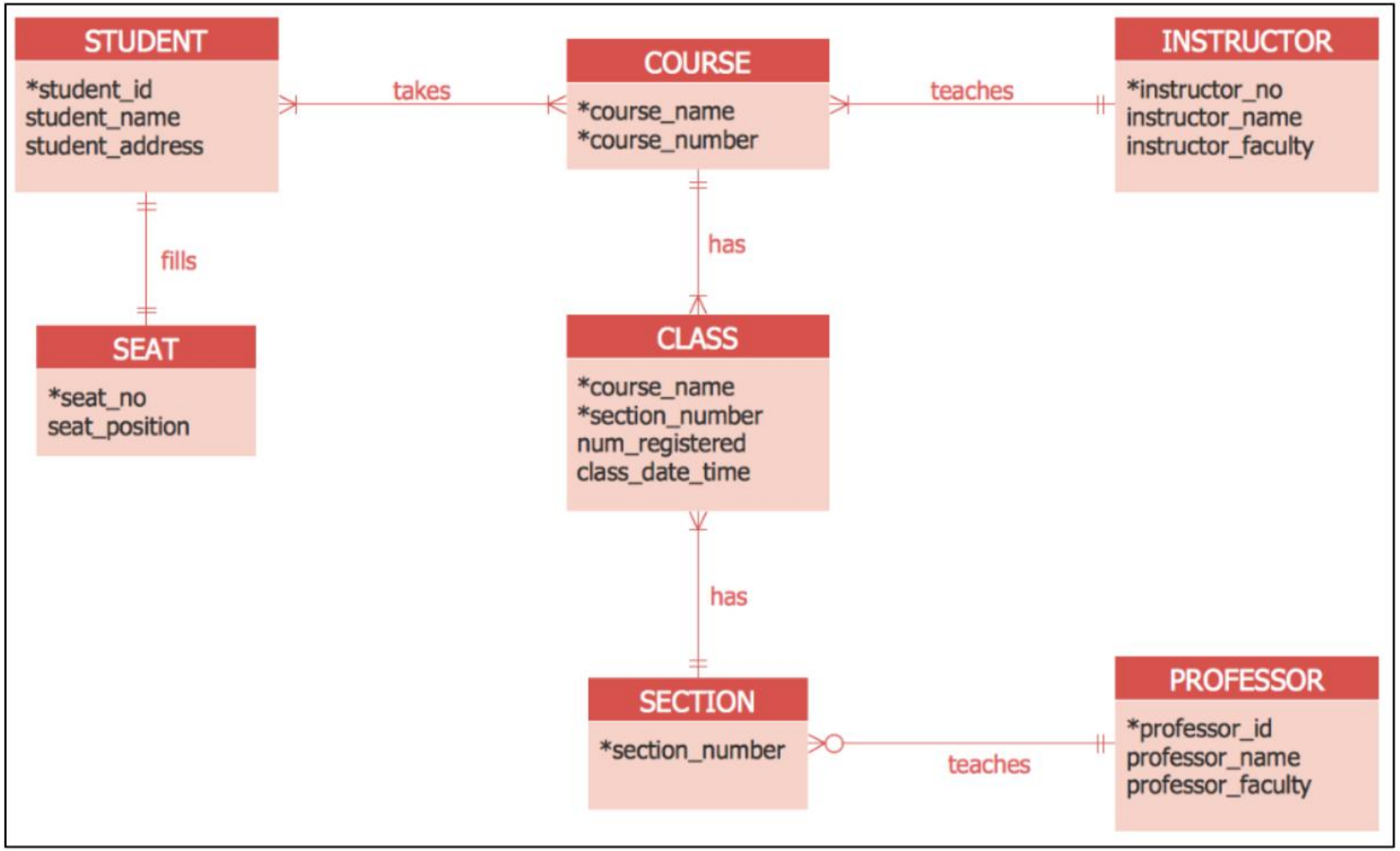
Step 5: Define Cardinality Mark each relationship with proper cardinality:

- One-to-Many (1:M): Use crow's foot notation (--|<)
- Many-to-Many (M:N): Requires junction table
- One-to-One (1:1): Single line (---|)

Step 6: Verify Relationships Double-check each relationship matches the business rules:

- "A salesperson may manage many other salespeople" ✓
- "A salesperson is managed by only one salesperson" ✓
- "A salesperson can be an agent for many customers" ✓
- "A customer is managed by one salesperson" ✓
- "A customer can place many orders" ✓
- "An order can be placed by one customer" ✓
- "An order lists many inventory items" ✓
- "An inventory item may be listed on many orders" ✓
- "An inventory item is assembled from many parts" ✓
- "A part may be assembled into many inventory items" ✓
- "Many employees assemble an inventory item" ✓
- "A supplier supplies many parts" ✓
- "A part may be supplied by many suppliers" ✓

Develop bi-directional complete business rules for each relationship between entities using below ERD.



- **STUDENT takes COURSE**
- A student can take many courses.
- A course can be taken by many students.
- **INSTRUCTOR teaches COURSE**
- An instructor can teach many courses.
- A course is taught by only one instructor.
- **COURSE has CLASS**
- A course can have many classes.
- A class belongs to only one course.
- **CLASS has SECTION**
- A class can have multiple sections.
- A section belongs to only one class.
- **SECTION teaches PROFESSOR**
- A professor can teach multiple sections.
- A section is taught by only one professor.
- **STUDENT fills SEAT**
- A student fills only one seat.
- A seat is filled by only one student.

Suppose you are given the following requirements for a simple database for the National Hockey League (NHL):

- the NHL has many teams,
- each team has a name, a city, a coach, a captain, and a set of players,
- each player belongs to only one team,
- each player has a name, a position (such as left wing or goalie), a skill level, and a set of injury records,
- a team captain is also a player,
- a game is played between two teams (referred to as host_team and guest_team) and has a date (such as May 11th, 2017) and a score (such as 4 to 2).

Construct a clean and concise ER diagram for the NHL database using the Chen notation as in your textbook. List your assumptions and clearly indicate the cardinality mappings as well as any role indicators in your ER diagram.

phanibhushan@crraoaimscs.res.in

Subject: DBMS-AssignmentNo.#

Entities & Attributes

1.TEAM

- Attributes: team_id (PK), team_name, city, coach
- Relationships:
  - Has many **players** (One-to-Many)
  - Has one **captain** (captain is a player)
  - Participates in **games** (host or guest)

2.PLAYER

- Attributes: player_id (PK), player_name, position, skill_level
- Relationships:
  - Belongs to **one team** (Many-to-One)
  - Can be a **captain** of one team (captain is a player)
  - Has multiple **injury records** (One-to-Many)

3.INJURY_RECORD

- Attributes: record_id (PK), injury_date, injury_description
- Relationships:
  - Belongs to **one player** (Many-to-One)

4.GAME

- Attributes: game_id (PK), game_date, host_score, guest_score
- Relationships:
  - Played between two **teams** (host_team, guest_team) (Two One-to-Many relations)

Chen Notation Considerations

- **Cardinality**:
  - One **team** has **many players** (1:N)
  - One **player** has **one team** (N:1)
  - A **game** involves **two teams** (host_team and guest_team) (1:N twice)
  - A **player** can have **multiple injury records** (1:N)
  - A **team** has **one captain**, who is a **player** (1:1)

Assumptions

1. Each **team** has only **one captain**, and the captain is a player.
2. Each **player** plays for only **one team** at a time.
3. A **game** always has **one host team and one guest team**.
4. The **game score** is stored as host_score and guest_score.

Introduction to the Relational Model: Integrity constraint over relations, enforcing integrity constraints, querying relational data, logical database design, introduction to views, destroying/altering tables and views.

Constraints enforce limits to the data or restrictions on data that can be inserted/updated/deleted from a table.

The whole purpose of constraints is to maintain the data integrity during an update/delete/insert into a table.

Integrity Constraints (ICs)

IC: condition that must be true for any instance of the database; e.g., domain constraints.

ICs are specified when schema is defined.

ICs are checked when relations are modified.

A legal instance of a relation is one that satisfies all specified ICs.

DBMS should not allow illegal instances.

If the DBMS checks ICs, stored data is more faithful to real-world meaning.

Avoids data entry errors, too!

Where do ICs Come From?

ICs are based upon the semantics of the real-world that is being described in the database relations.

We can check a database instance to see if an IC is violated, but we can NEVER infer that an IC is true by looking at an instance.

- An IC is a statement about all possible instances!
- From example, we know name is not a key, but the assertion that sid is a key is given to us.

Key and foreign key ICs are the most common; more general IC s supported too.

Constraints on databases can generally be divided into three main categories:

1. Constraints those are inherent in the data model, we call these inherent model-based constraints or implicit constraints.

2. Constraints that can be **directly expressed in schemas** of the data model, typically by specifying them in the DDL.

3. Constraints that cannot be directly expressed in the schemas of the data model, and hence must be expressed and enforced by the application programs. This is known as application-based or semantic constraints or business rules.

The **schema-based** constraints include:

- domain constraints,
- key constraints,
- entity integrity constraints and
- referential integrity constraints.

Domain Constraints

Each table has certain set of columns and each column allows the same type of data based on its data type. The column does not accept values of any other data type. Domain constraints can be defined as follows:

Domain Constraint = data type + Constraints (NOT NULL /UNIQUE / PRIMARY KEY / FOREIGNKEY / CHECK /DEFAULT).

ROLL	NAME	AGE
1	Rahul Sarmah	23
2	Smriti Gogoi	22
3	Shahidul Khan	24
4	Rupak Chetri	A

Here, value A is not allowed since only integer values can be taken by the age attribute.

Key Constraints

An attribute that can uniquely identify a tuple in a relation is called the key of the table. All the values in the primary key column must be unique.

ROLL	NAME	AGE
1	Rahul Sarmah	23
1	Smriti Gogoi	22
3	Shahidul Khan	24
4	Rupak Chetri	22

This relation/table does not satisfy the key constraint as here all the values of primary key are not unique.

Entity Integrity Constraint

Entity integrity constraint specifies that in a relation no attribute value of primary key attribute must contain a null value. This is because the presence of null value in the primary key violates the uniqueness property.

ROLL	NAME	AGE
1	Rahul Sarmah	23
2	Smriti Gogoi	22
3	Shahidul Khan	24
	Rupak Chetri	22

This relation does not satisfy the entity integrity constraint as here the primary key contains a null value.

Entity Integrity Constraint - Formally

- **Relational Database Schema**: A set S of relation schemas that belong to the same database. S is the *name* of the **database**

$$S = \{R_1, R_2, ..., R_n\}$$

- **Entity Integrity**: The *primary key attributes* PK of each relation schema R in S cannot have null values in any tuple of r(R). This is because primary key values are used to *identify* the individual tuples.

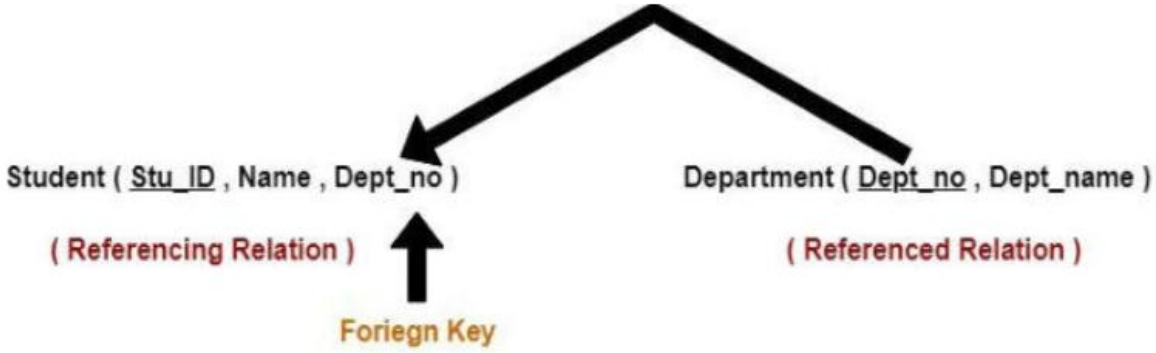
$$t[PK] \neq \text{null for any tuple t in r(R)}$$

- Note: Other attributes of R may be similarly constrained to disallow null values, even though they are not members of the primary key

Referential Integrity Constraint

Referential Integrity constraints work on the concept of Foreign Keys. A foreign key is an attribute of a relation which must be a primary key or the part of the primary key in another relation

A set of attributes FK in relation schema R 1 is a foreign key of R 1 that references relation R 2 if it satisfies. R 1 is called the referencing relation and R 2 is the referenced relation.



STUDENT Relation (Referencing Relation)

Roll	Name	Dept_No
1	Rahul Sarmah	D1
2	Smriti Gogoi	D1
3	Shahidul Khan	D2
4	Rupak Chetri	D14

DEPARTMENT Relation (Referenced Relation)

Dept_No	Dept_Name
D1	Mathematics
D2	Physics
D3	Chemistry
D5	Computer Science

From the tables it is clear that,

- The relation 'Student' does not satisfy the referential integrity constraint.
- This is because in relation 'Department', no value of primary key specifies department no.14.
- Thus, referential integrity constraint is violated.

Referential Integrity Constraint – Formal Definition

- A constraint involving *two* relations (the previous constraints involve a *single* relation)
- Used to specify a *relationship* among tuples in two relations: the **referencing relation** and the **referenced relation**
- Tuples in the *referencing relation* R_1 have attributes FK (called **foreign key** attributes) that reference the primary key attributes PK of the *referenced relation* R_2. A tuple t_1 in R_1 is said to **reference** a tuple t_2 in R_2 if $t_1[FK] = t_2[PK]$
- A referential integrity constraint can be displayed in a relational database schema as a directed arc from R_1.FK to R_2

Enforcing Referential Integrity

Consider Students and Enrolled; sid in Enrolled is a foreign key that references Students.

- What should be done if an Enrolled tuple with a non-existent student id is inserted?
  - (Reject it!)
- What should be done if a Students tuple is deleted?
  - Also delete all Enrolled tuples that refer to it.
  - Disallow deletion of a Students tuple that is referred to.
  - Set sid in Enrolled tuples that refer to it to a default sid.
  - (In SQL, also: Set sid in Enrolled tuples that refer to it to a special value null, denoting `unknown' or `inapplicable'.)
- Similar if primary key of Students tuple is updated

Example Tables:

Students Table (Referenced Relation)

sid	name	major
101	Alice	CS
102	Bob	Math
103	Charlie	Physics

Enrolled Table (Referencing Relation)

enroll_id	sid	course
1	101	AI
2	102	ML
3	101	DBMS
4	103	Networks

Here, sid in **Enrolled** is a **foreign key** referencing sid in **Students**.

Options for Handling Deletions

1.Cascade Delete (ON DELETE CASCADE)

- **Effect:** When a student is deleted from Students, all Enrolled tuples referring to that sid are also deleted.

- **Resulting Tables:**
- **Students Table (After Deleting sid=101)**

sid	name	major
102	Bob	Math
103	Charlie	Physics

- **Enrolled Table (After Deleting sid=101)**

enroll_id	sid	course
2	102	ML
4	103	Networks

Use Case: When enrollment records should be removed along with student removal.

Example Deletion Strategies

sid	name	login	age	gpa
~~53666~~	~~Jones~~	~~jones@cs~~	~~18~~	~~3.4~~
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

sid	cid	grade
53831	Carnatic101	C
53831	Reggae203	B
53650	Topology112	A
~~53666~~	~~History105~~	~~B~~

sid	cid	grade
53831	Carnatic101	C
53831	Reggae203	B
53650	Topology112	A
NULL	History105	B

Example Update Strategies

sid	name	login	age	gpa
55666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

sid	cid	grade
53831	Carnatic101	C
53831	Reggae203	B
53650	Topology112	A
NULL	History105	B

sid	cid	grade
53831	Carnatic101	C
53831	Reggae203	B
53650	Topology112	A
99999	History105	B

Relational Integrity Constraints

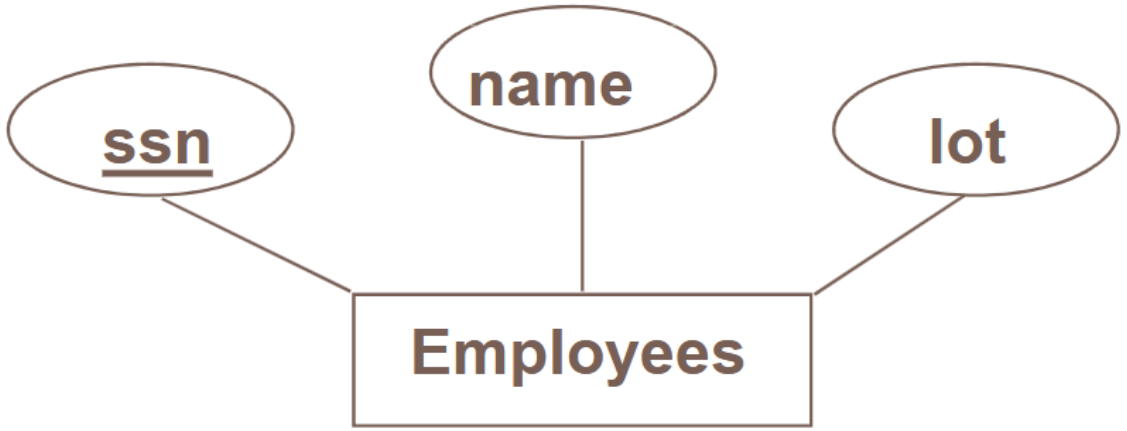
Other Types of Constraints

- Semantic Integrity Constraints:
  - based on application semantics and cannot be expressed by the model per se
  - E.g., “the max. no. of hours per employee for all projects he or she works on is 56 hrs per week”
  - A *constraint specification language* may have to be used to express these
  - SQL-99 allows triggers and ASSERTIONS to allow for some of these

- Transition/dynamic constraints: e.g., “the salary of an employee can only increase”

Logical DB Design: ER to Relational

- Entity sets to tables.



ssn	name	lot
123-22-3666	Attishoo	48
231-31-5368	Smiley	22
131-24-3650	Smethurst	35

```
CREATE TABLE Employees
    (ssn CHAR(11),
    name CHAR(20),
    lot  INTEGER,
    PRIMARY KEY  (ssn))
```

Transforming the EER diagram into relations

The steps:

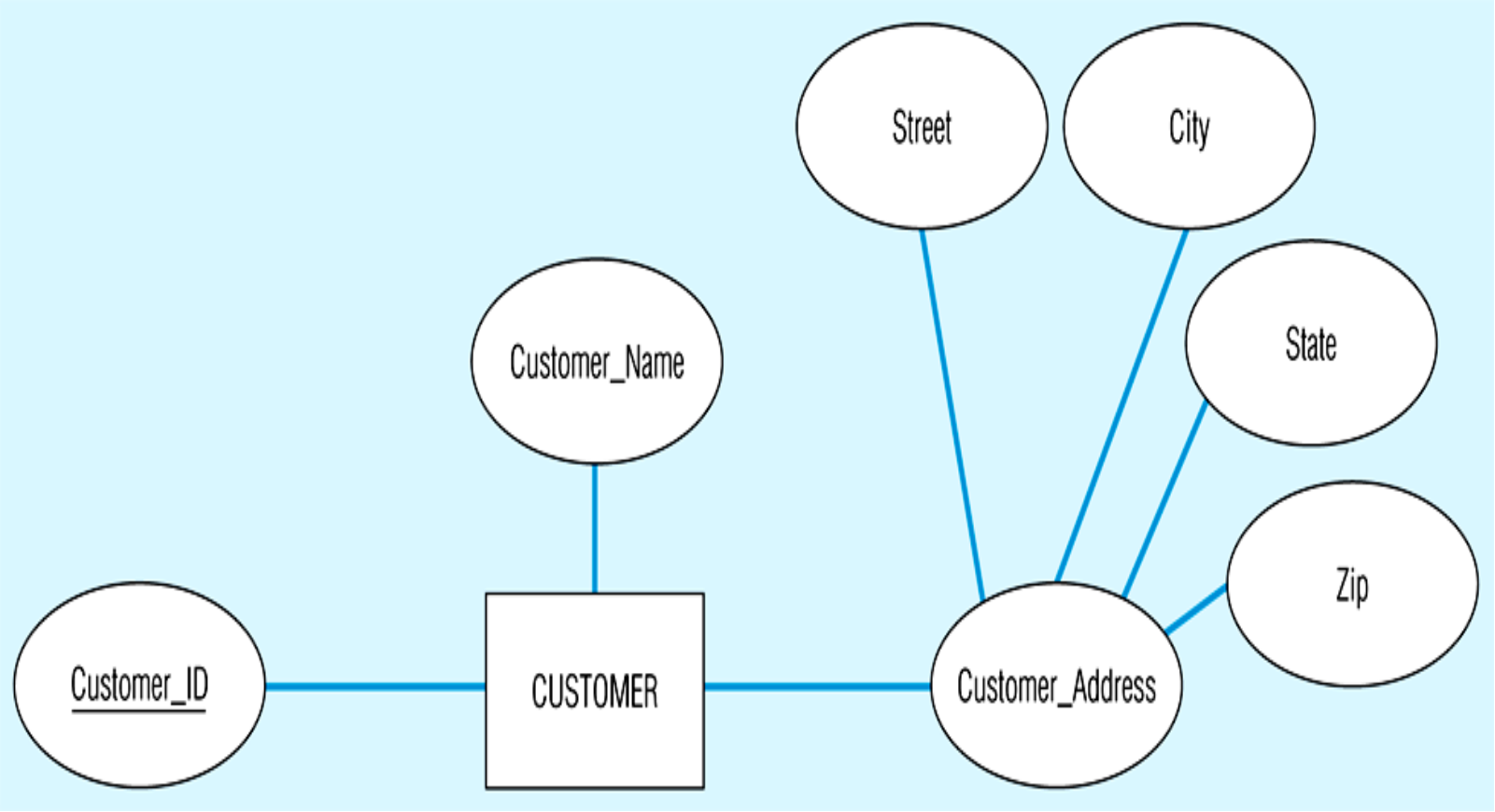
- Map regular entities
- Map weak entities
- Map binary relationships
- Map associative entities
- Map unary relationships
- Map ternary relationships
- Map supertype/subtype relationships

Transforming E-R diagrams into relations

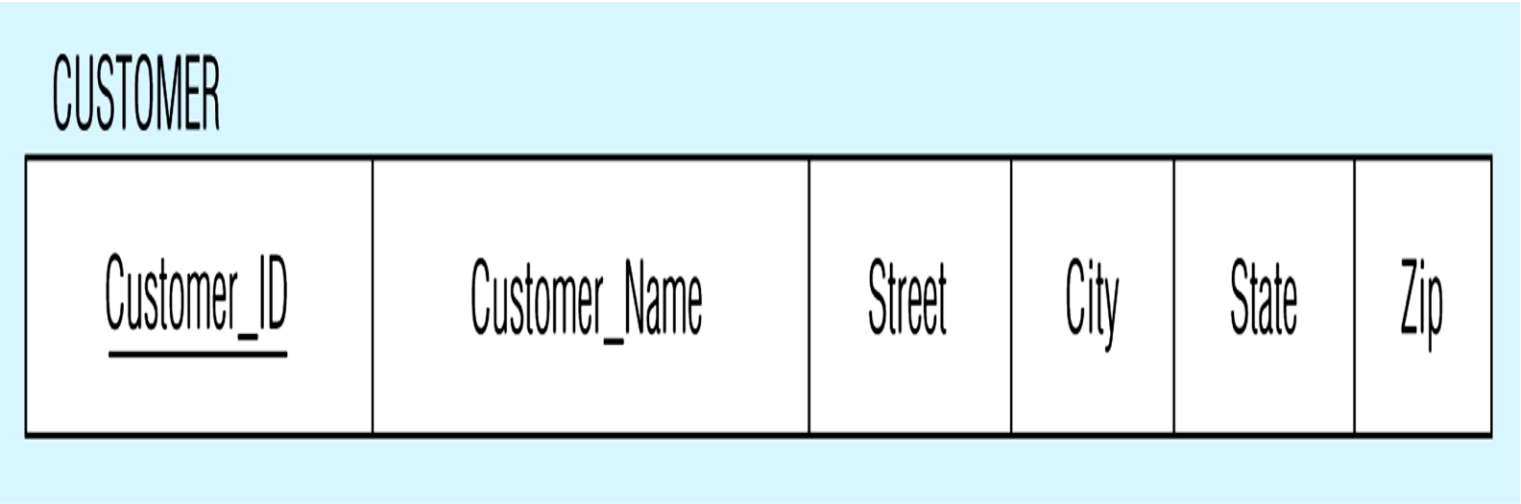
Mapping regular entities to relations

- Composite attributes: use only their simple, component attributes
- Multi-valued attributes: become a separate relation with a foreign key taken from the superior entity

Mapping a composite attribute



Looks like this using relational schema notation

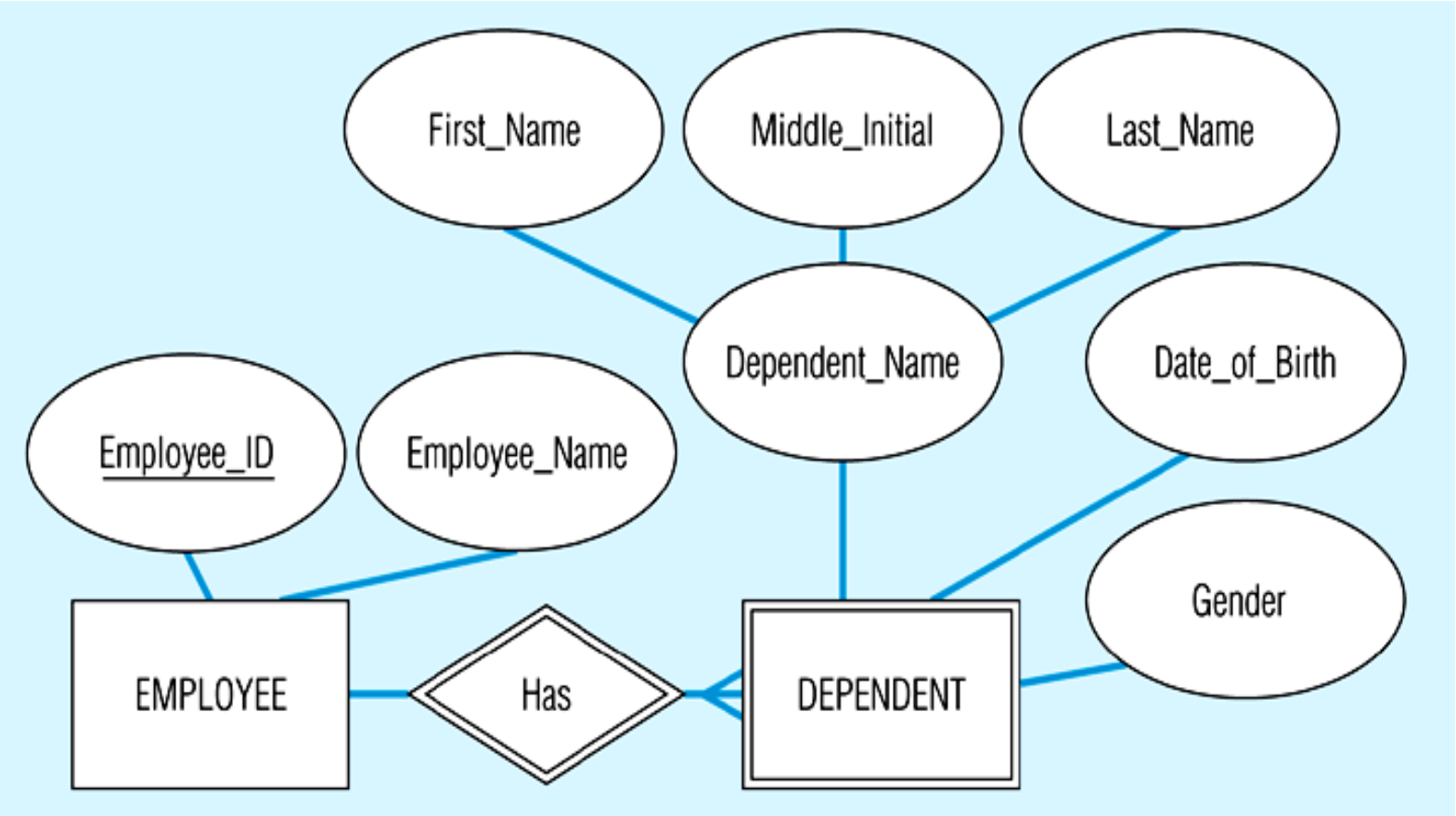


Transforming E-R diagrams into relations

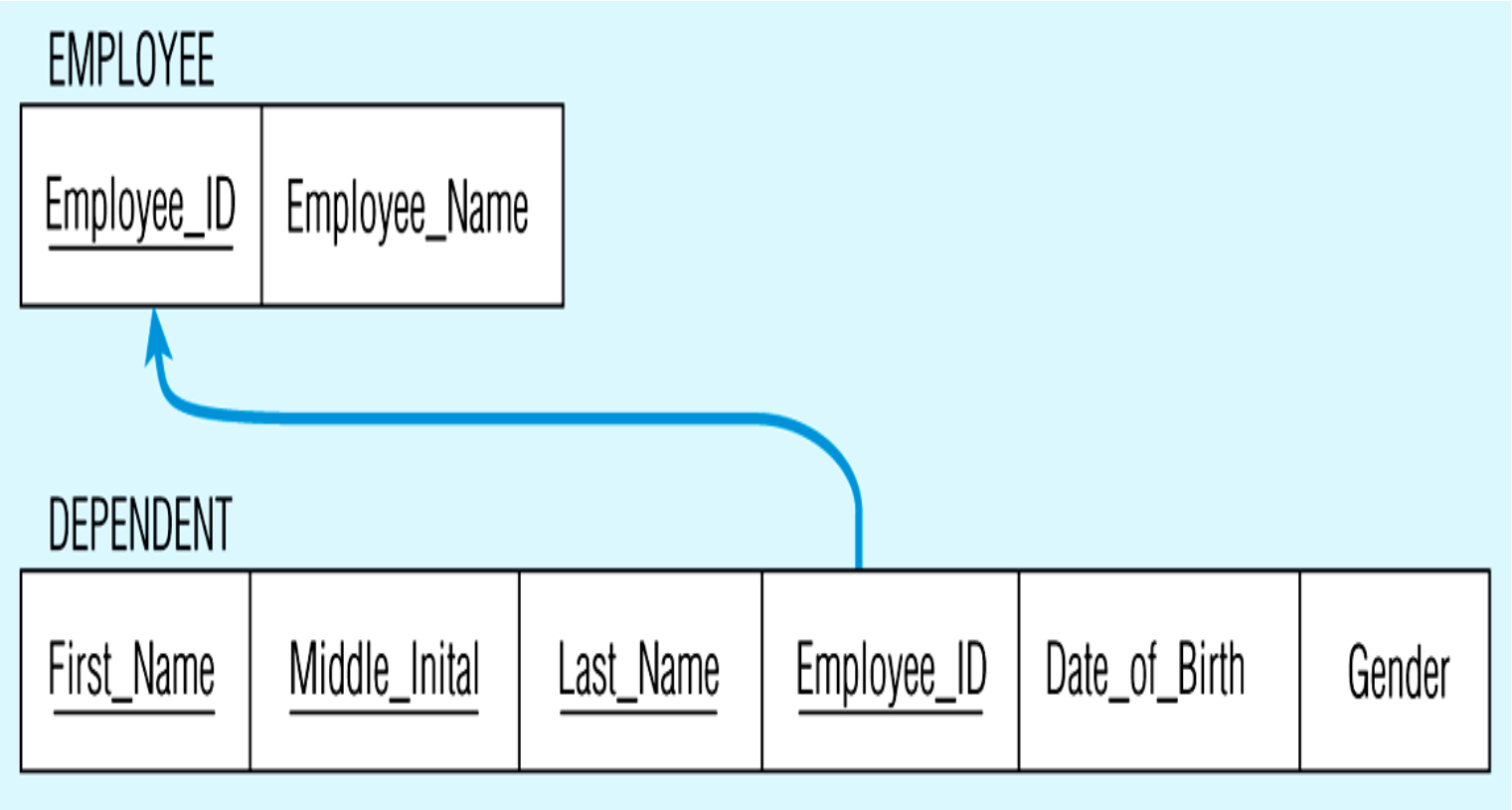
Mapping weak entities

- Becomes a separate relation with a foreign key taken from the superior entity

Example of mapping a weak entity



Looks like this using relational schema notation

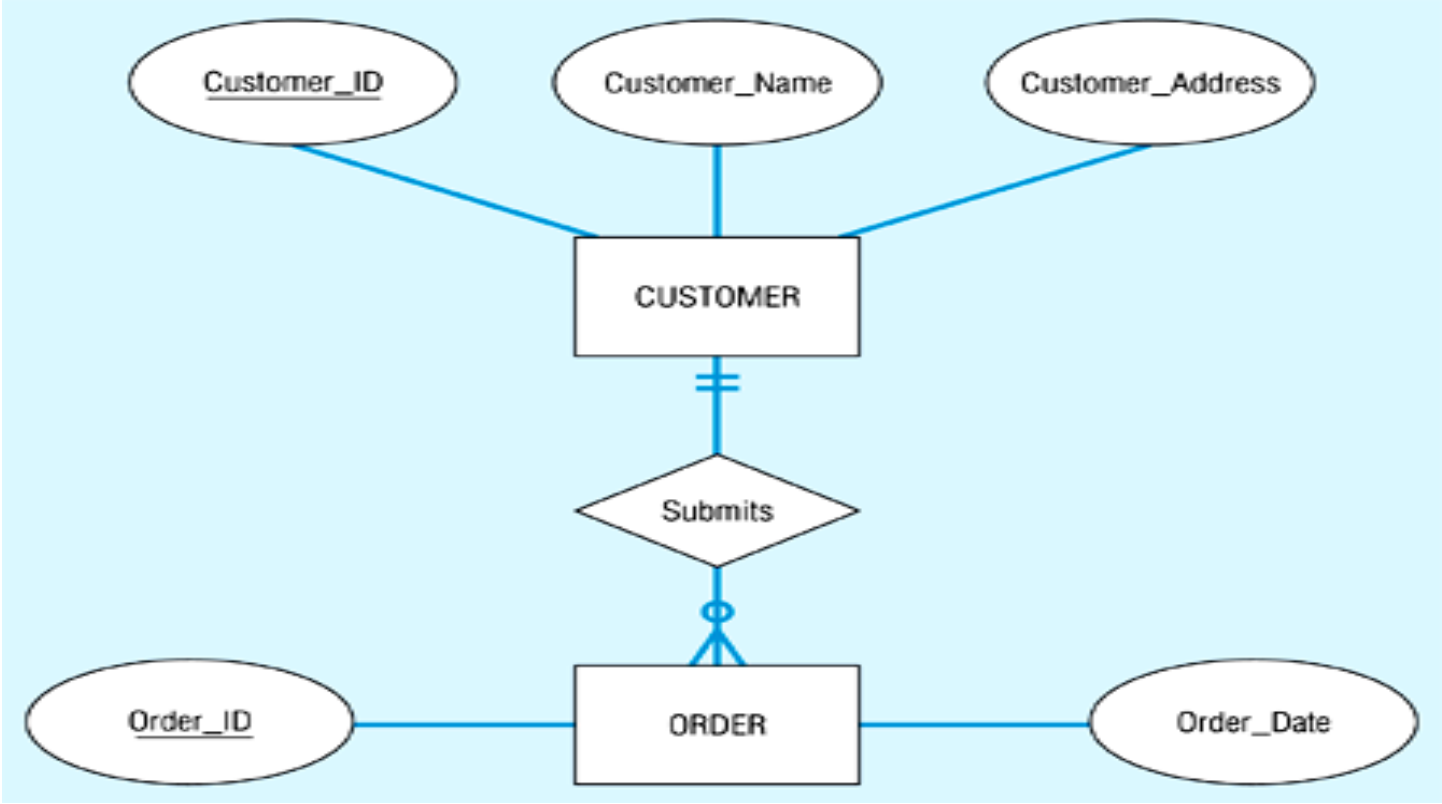


Transforming E-R diagrams into relations

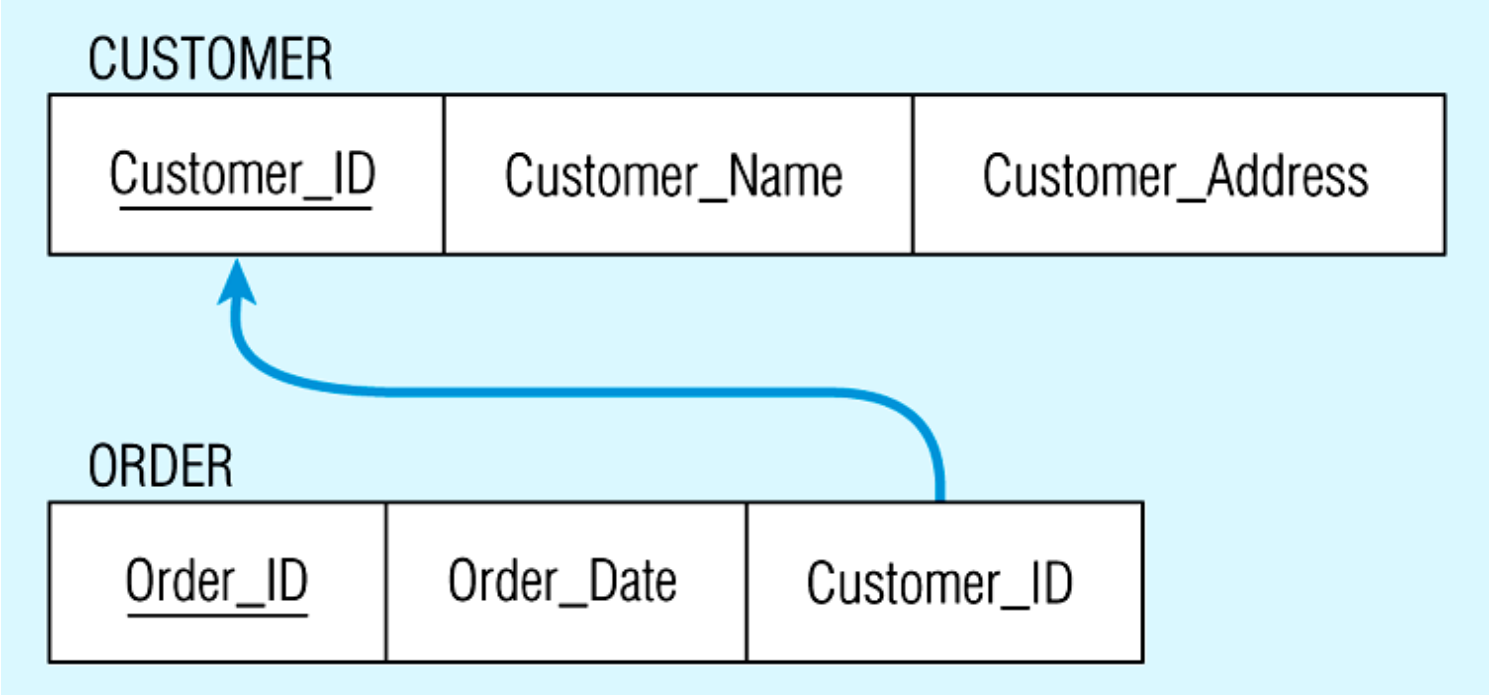
Mapping binary relationships

- **One-to-many - primary key on the one side becomes a foreign key on the many side**
- **Many-to-many - create a new relation (associative entity) with the primary keys of the two entities as its primary key**
- **One-to-one - primary key on the mandatory side becomes a foreign key on the optional side**

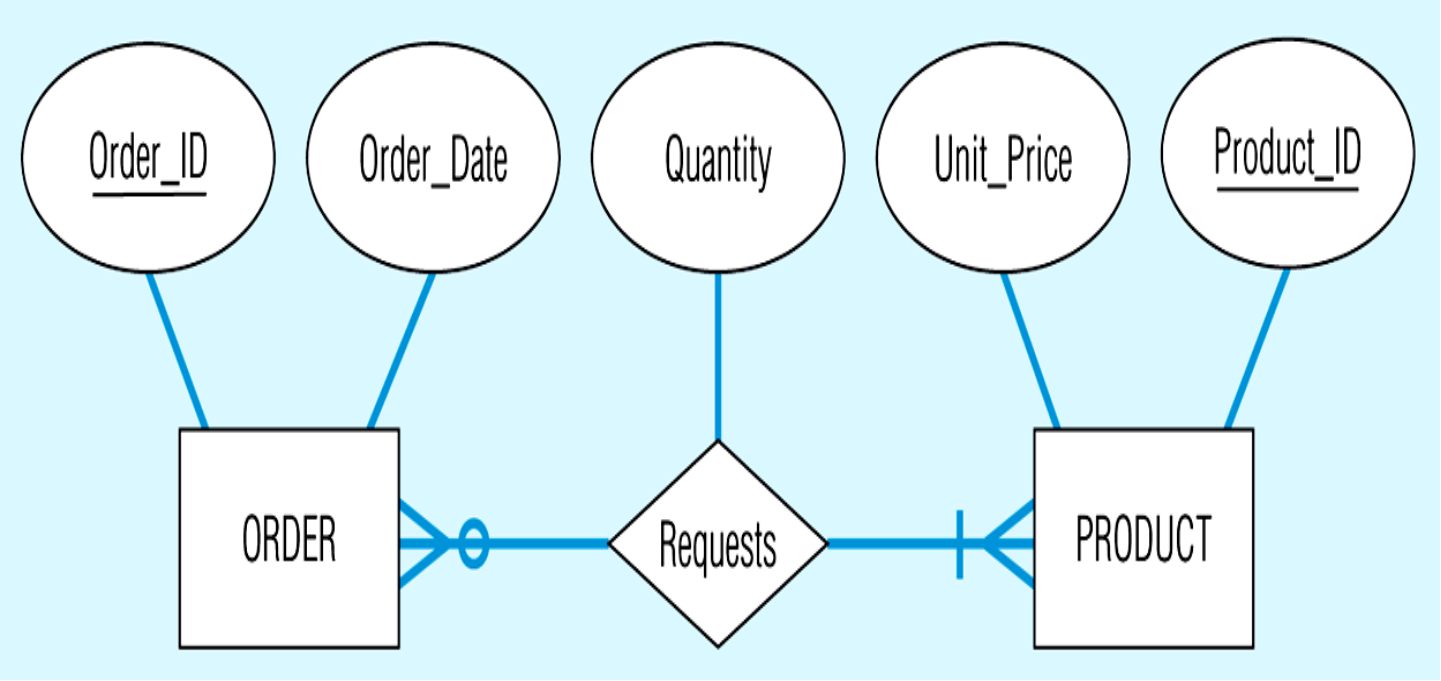
Example of mapping a 1:M relationship



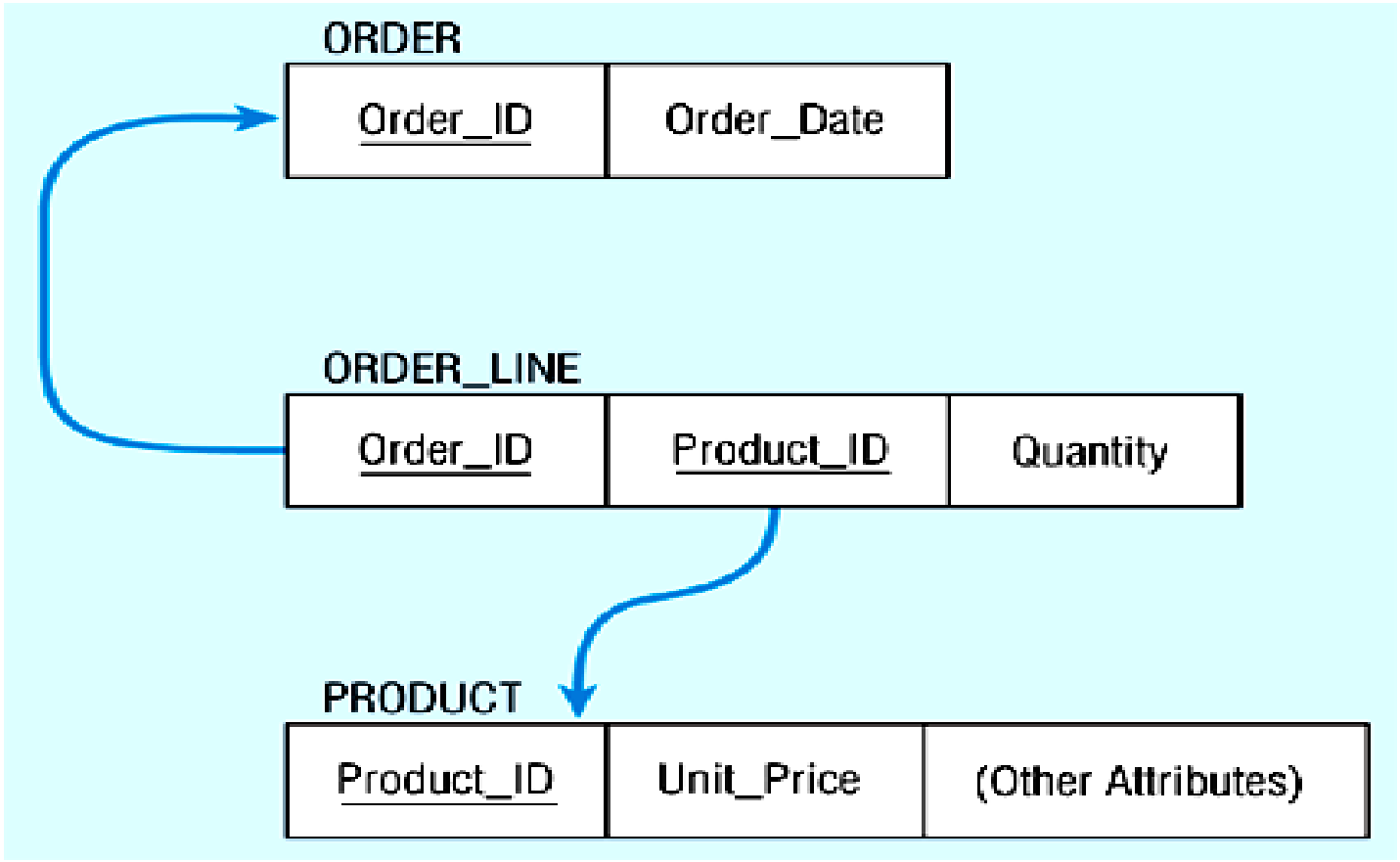
Looks like this using relational schema notation



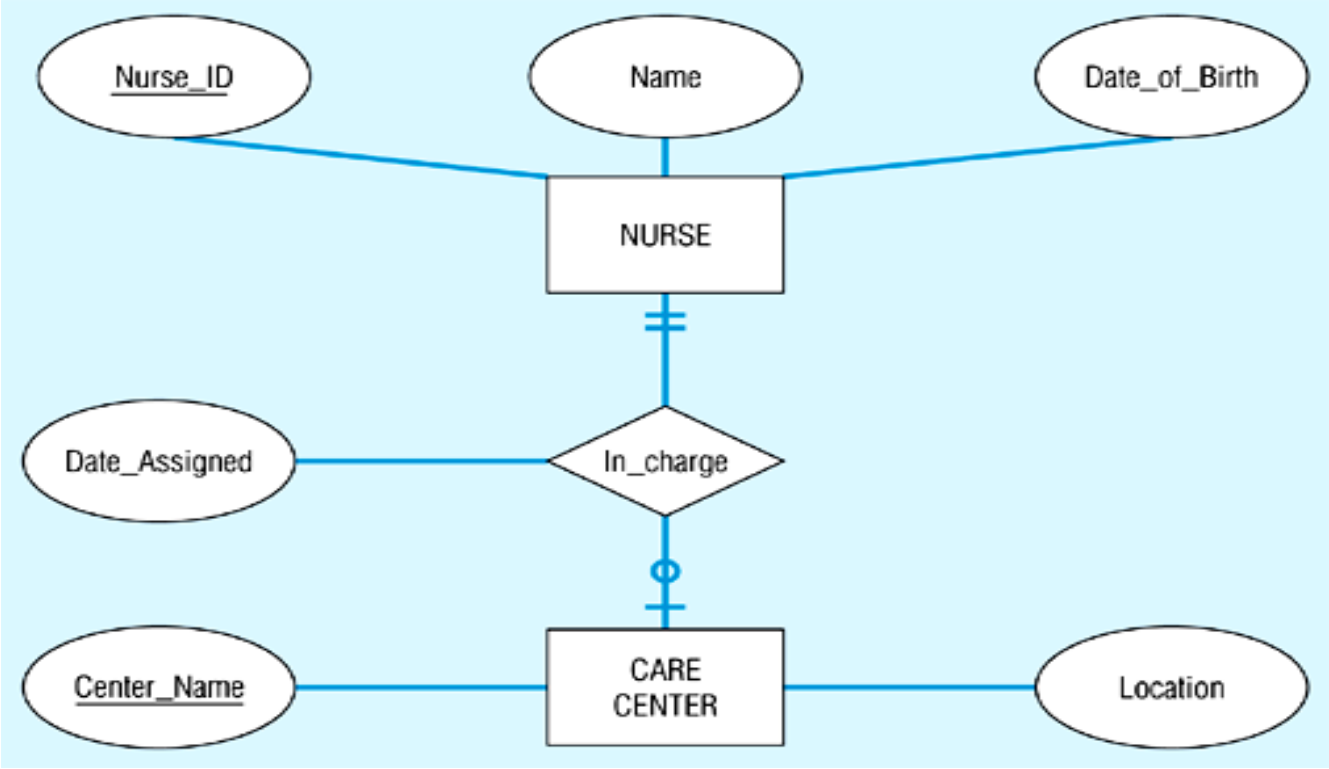
Example of mapping an M:M relationship



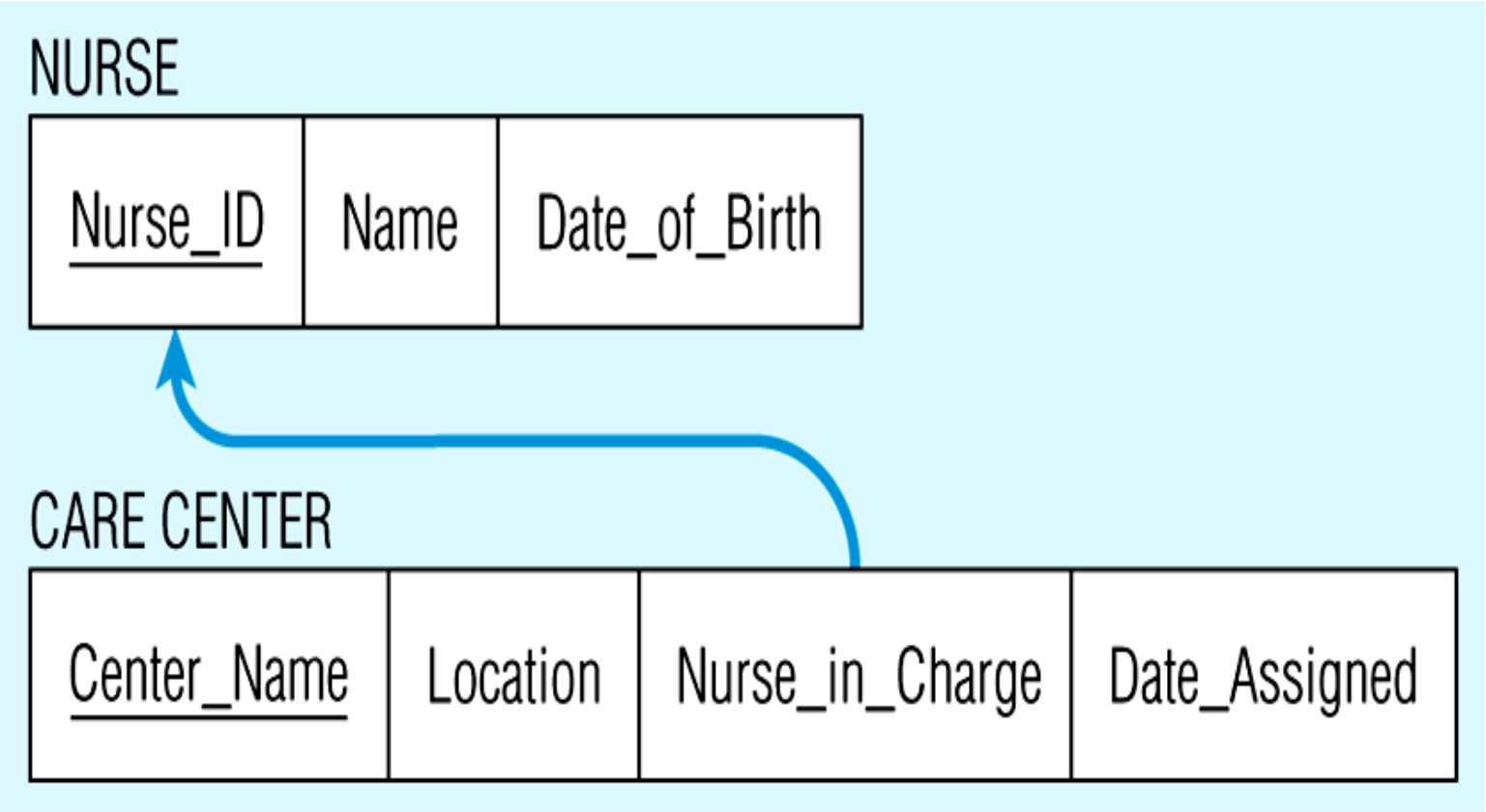
Looks like this using relational schema notation



Mapping a binary 1:1 relationship



Looks like this using relational schema notation

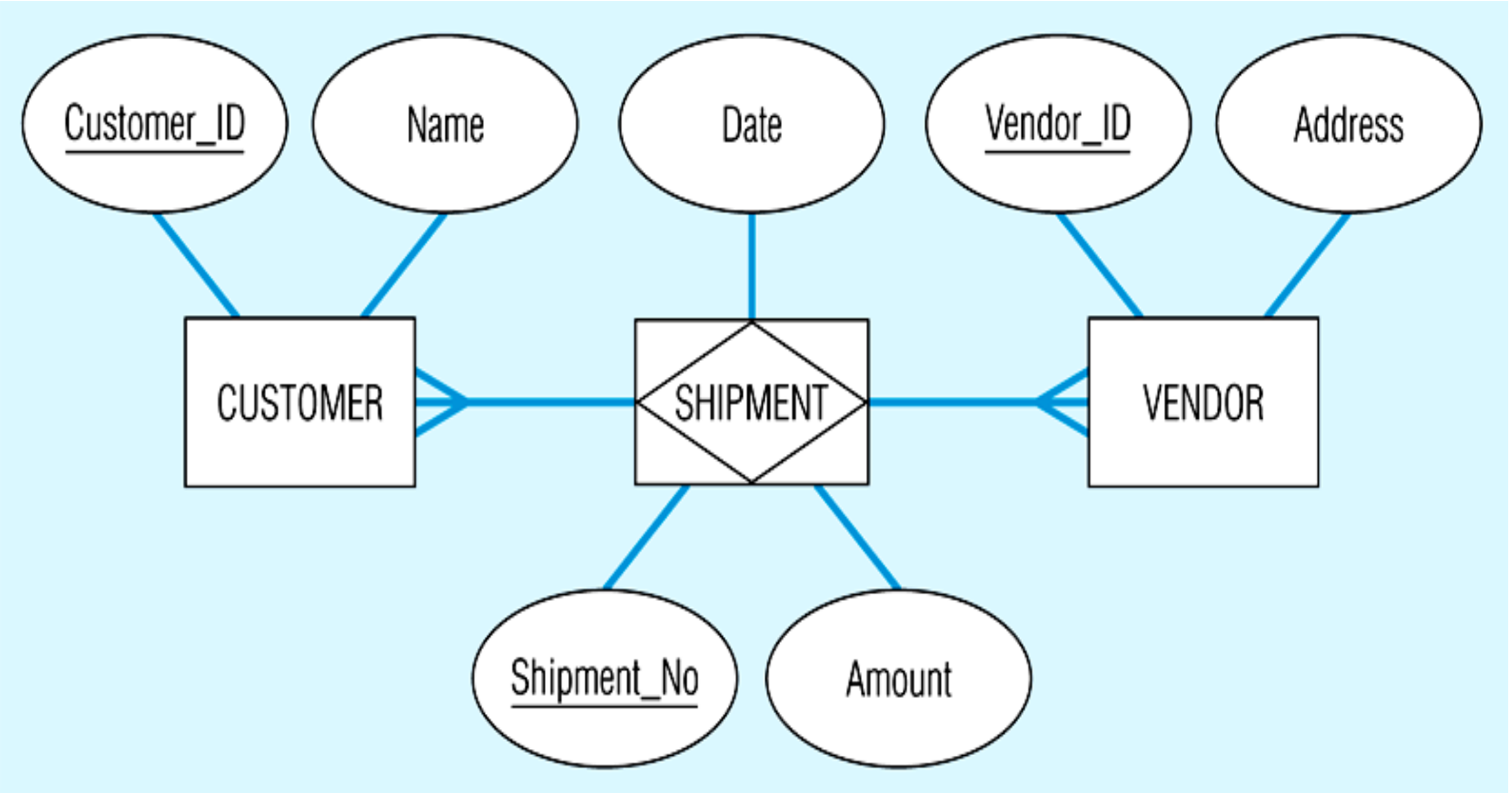


Transforming E-R diagrams into relations

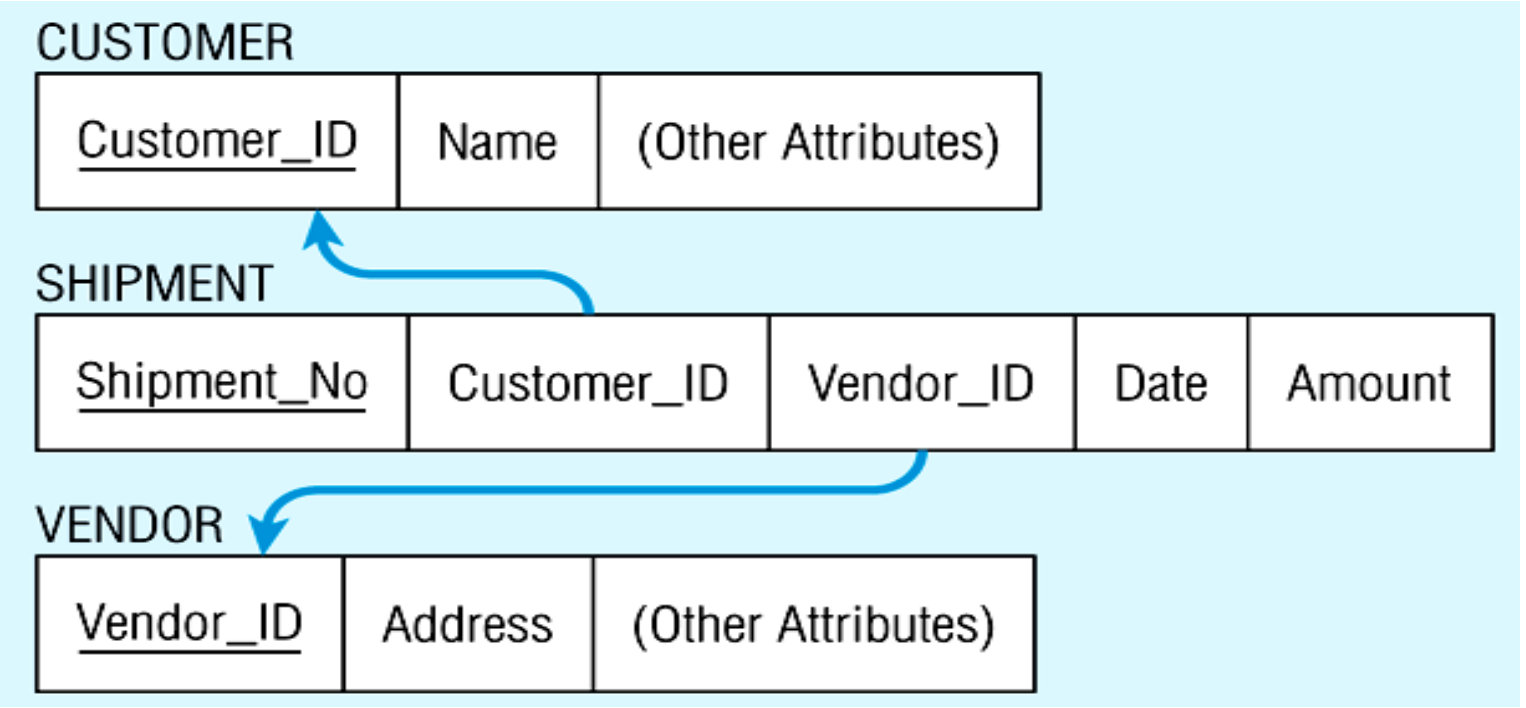
Mapping associative entities

- Identifier not assigned
  - Default primary key for the association relation is the primary keys of the two entities
- Identifier assigned
  - It is natural and familiar to end-users
  - Default identifier may not be unique

Mapping an associative entity with an identifier



Looks like this using relational schema notation

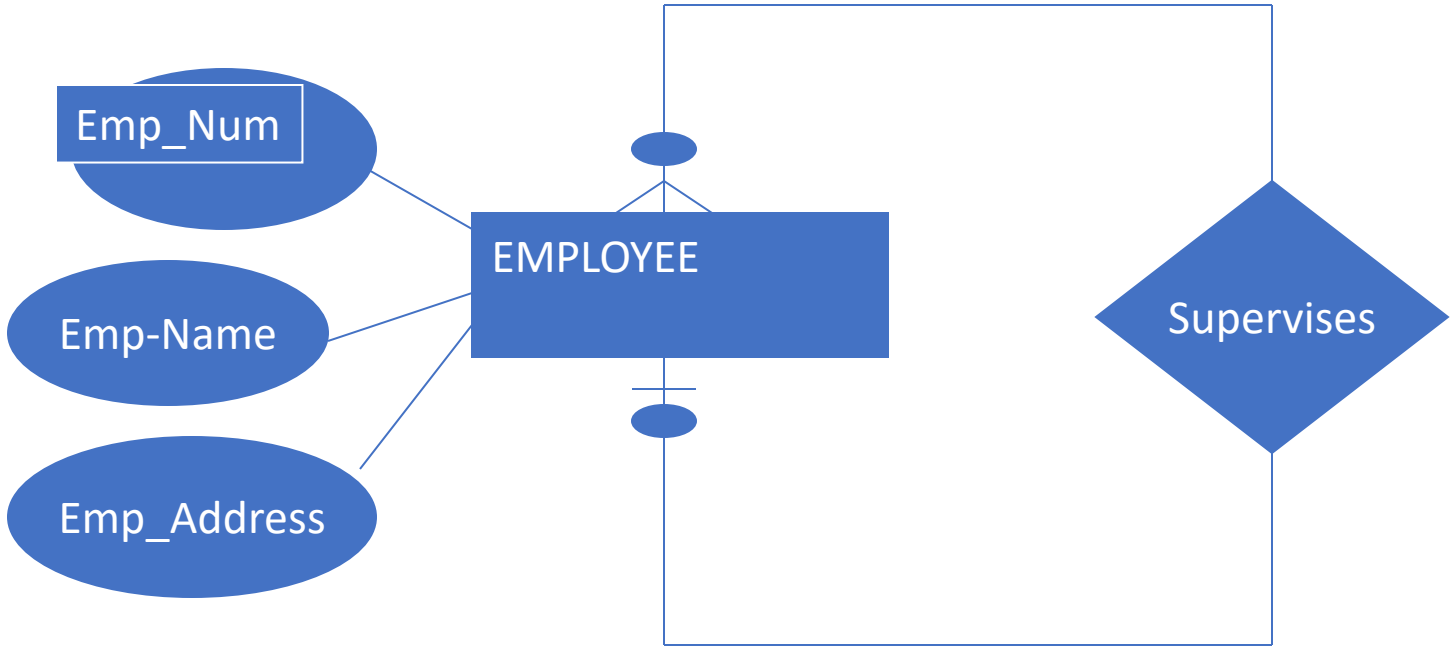


Transforming E-R diagrams into relations

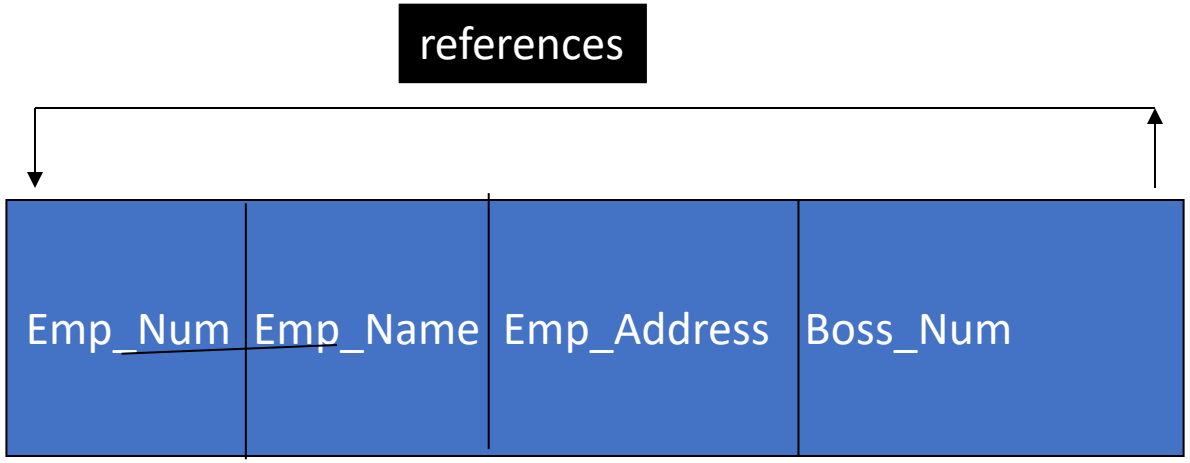
Mapping unary relationships

- One-to-many - recursive foreign key in the same relation
- Many-to-many - two relations:
  - One for the entity type
  - One for an associative relation in which the primary key has two attributes, both taken from the primary key of the entity

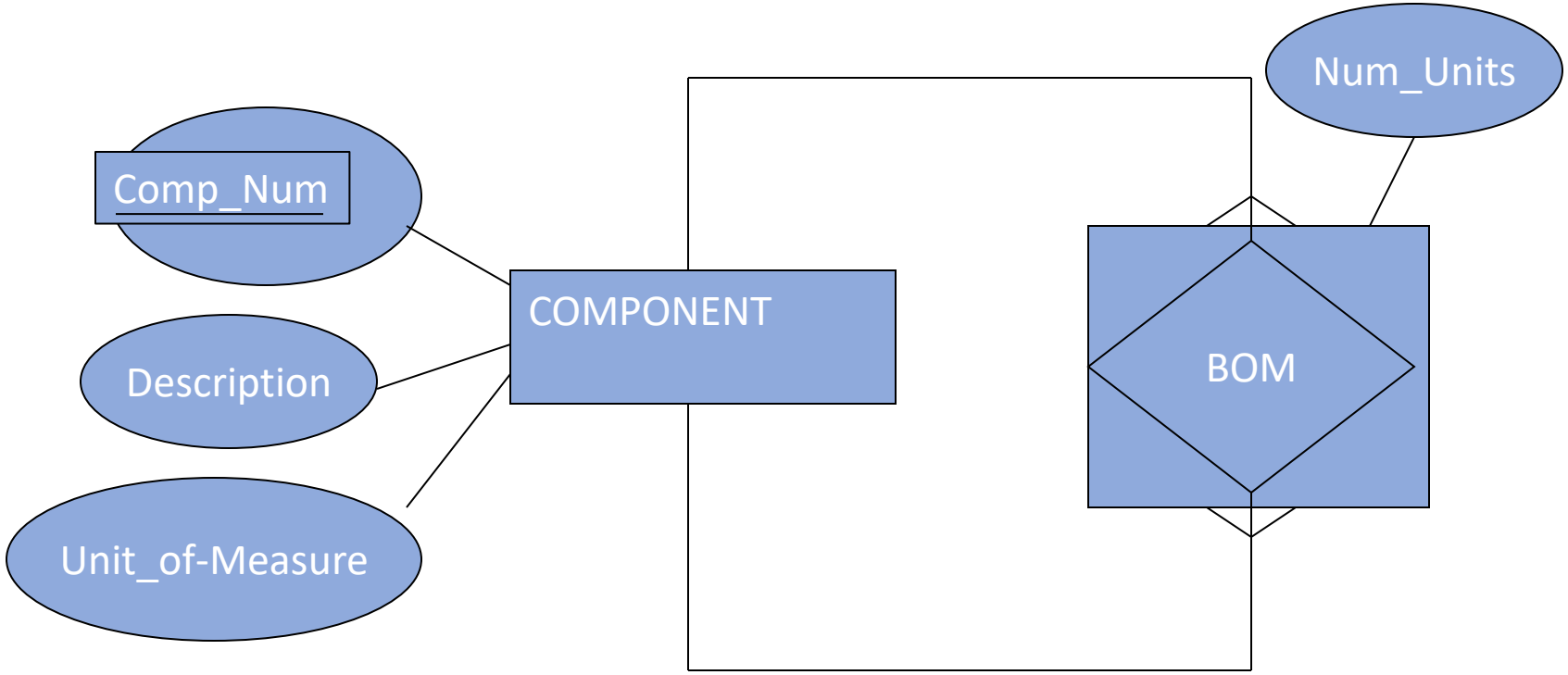
For example...



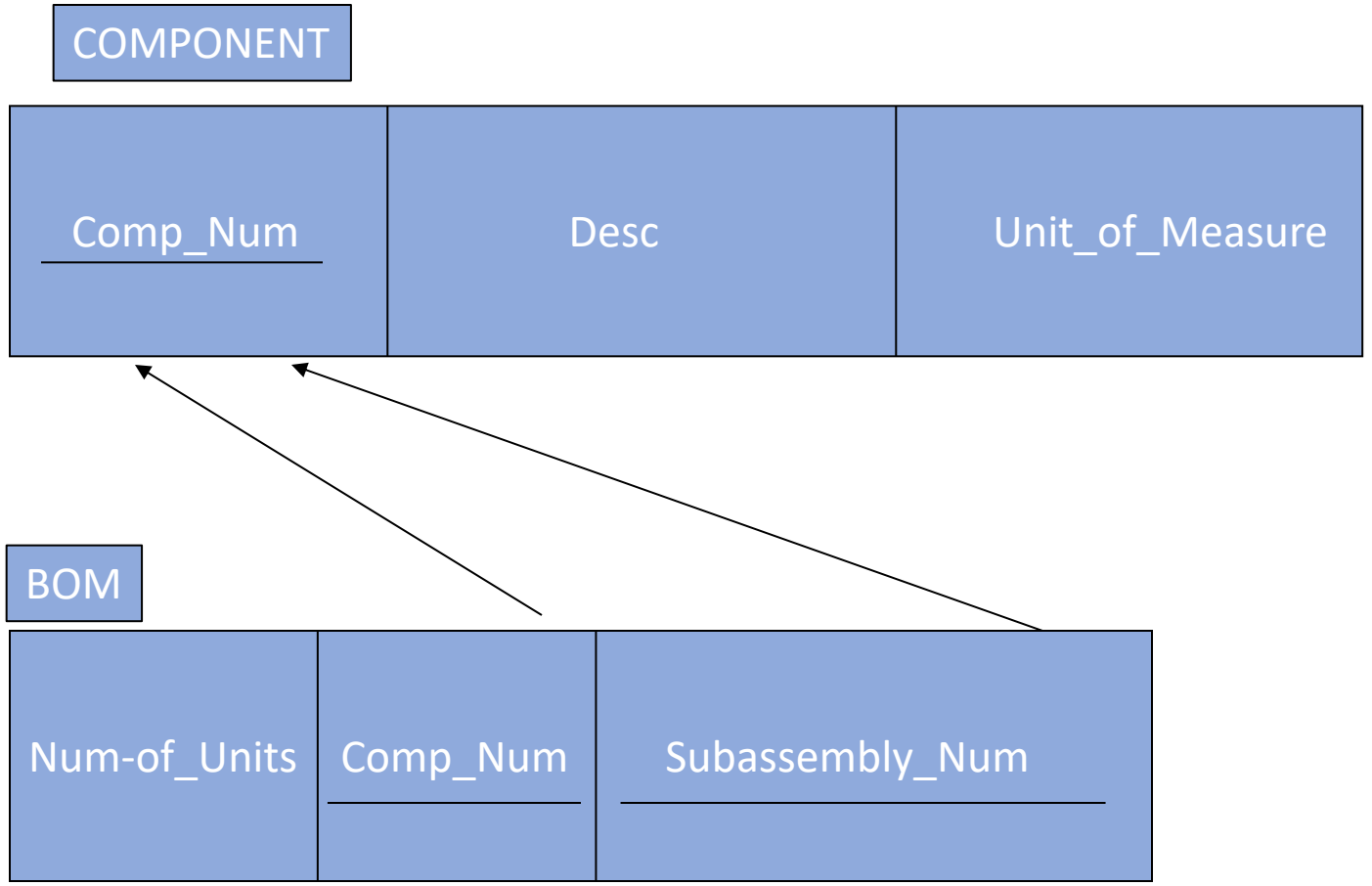
Would look like…



And..


Num_Units
Comp_Num
COMPONENT
Description
BOM
Unit_of-Measure

Would look like…

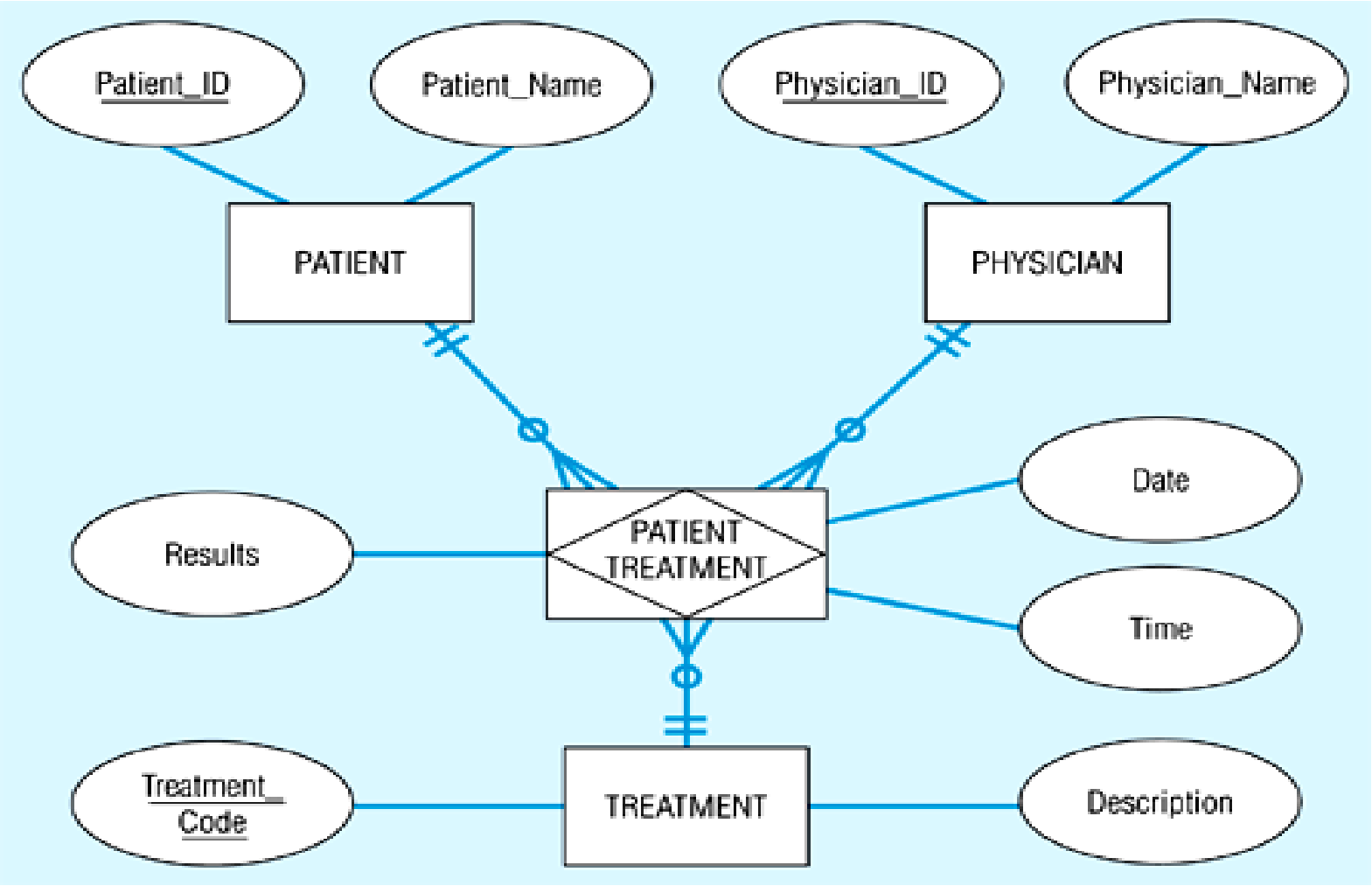


Transforming E-R diagrams into relations

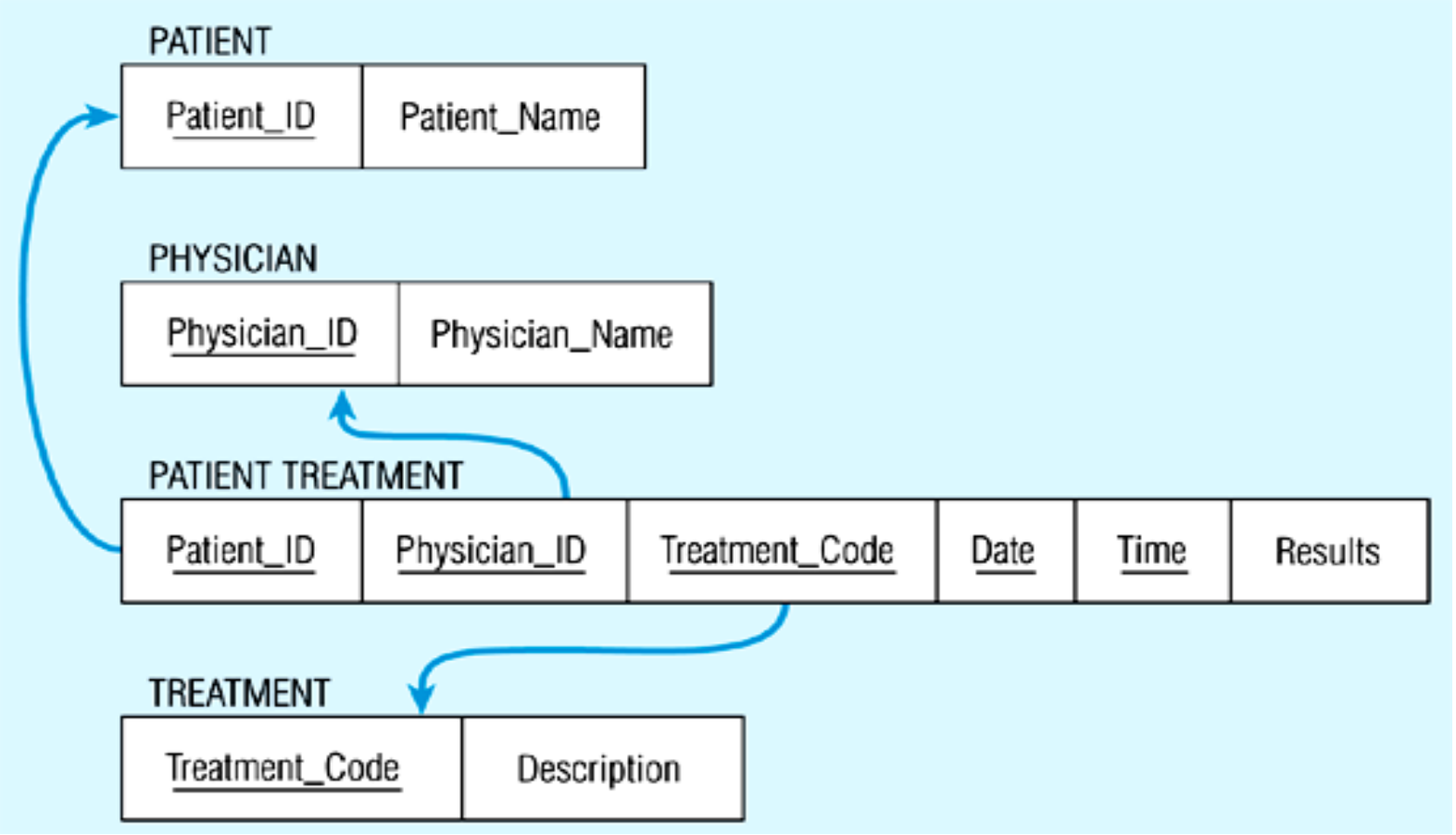
Mapping ternary (and n-ary) relationships

- One relation for each entity and one for the associative entity

Mapping a ternary relationship



Looks like this using relational schema notation

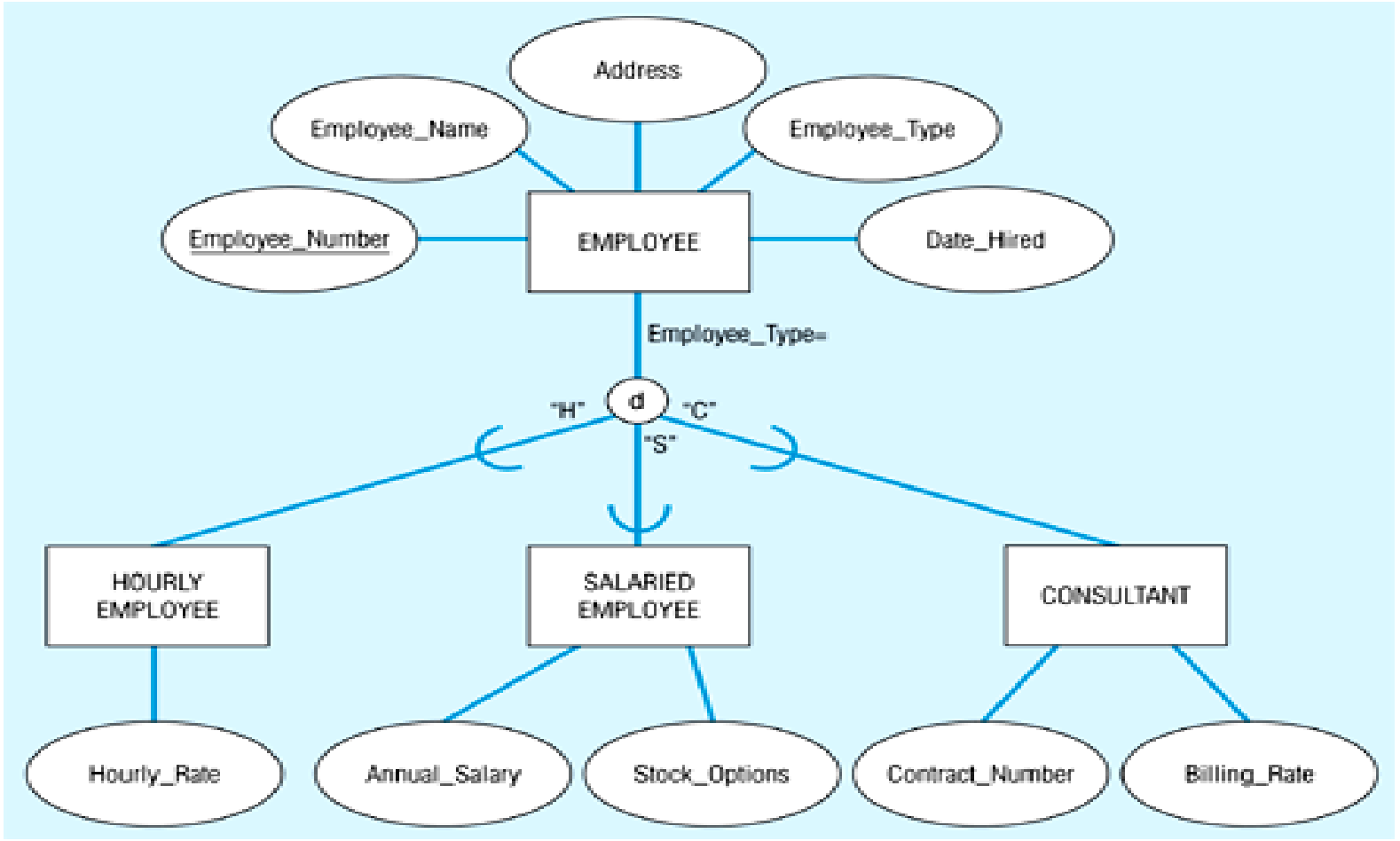


Transforming E-R diagrams into relations

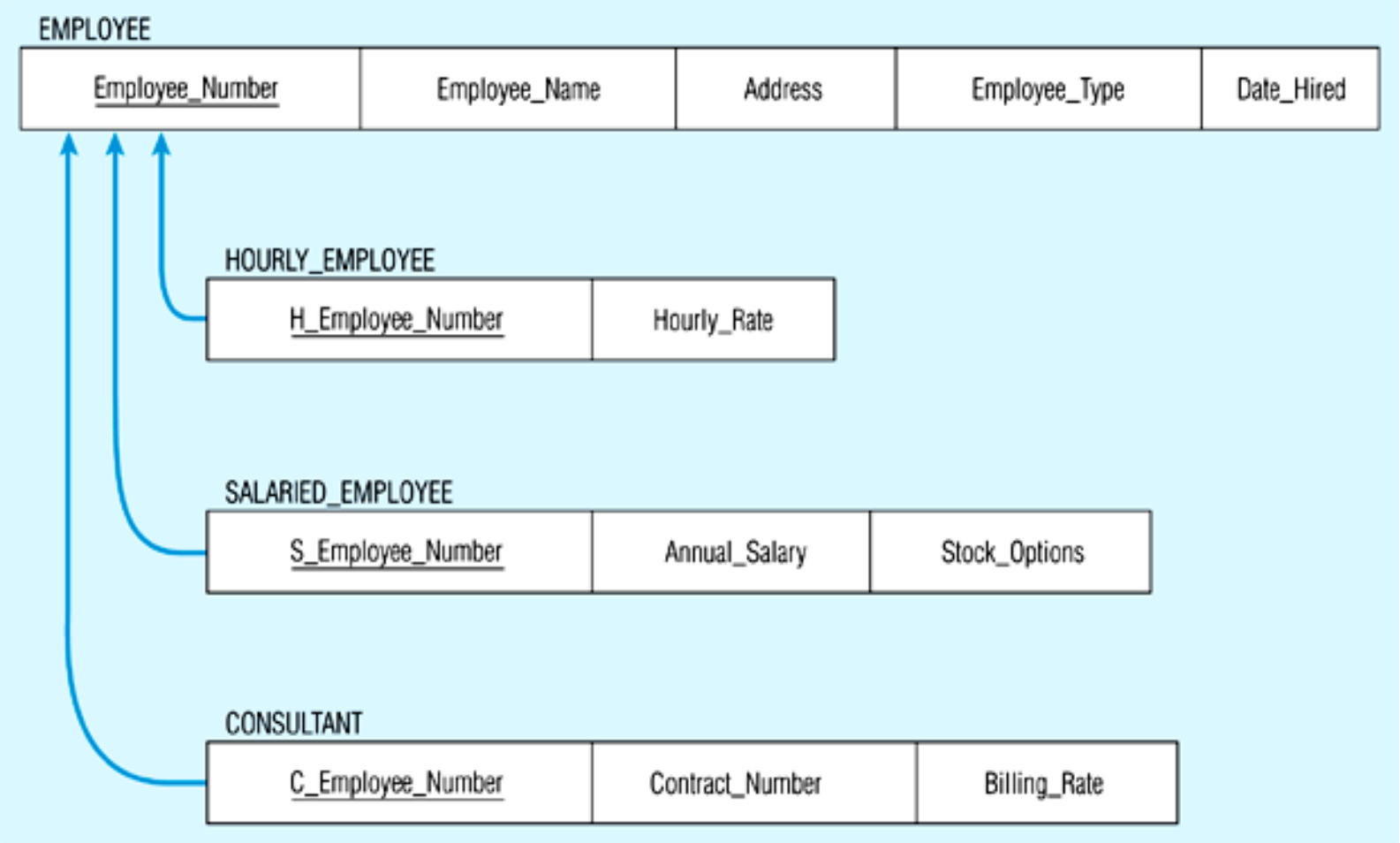
Mapping Supertype/subtype relationships

- Create a separate relation for the supertype and each of the subtypes
- Assign common attributes to supertype
- Assign primary key and unique attributes to each subtype
- Assign an attribute of the supertype to act as subtype discriminator

Mapping Supertype/subtype relationships



Would look like this…



VIEWS

Views provide virtual relations which contain data spread across different tables. Used by applications.

- simplified query formulations
- data hiding
- logical data independence

Once created, a view is always kept up-to-date by the RDBMS

View is not part of conceptual schema

- created to give a user group, concerned with a certain aspect of the information system, their view of the system

Storage

- Views need not be stored as permanent tables
- They can be created on-the-fly whenever needed
- They can also be materialized

Tables involved in the view definition –base tables

View Definition

A relation that is not of the conceptual model but is made visible to a user as a “virtual relation” is called a view.

A view is defined using the create view statement which has the form

create view v as < query expression >

where <query expression> is any legal SQL expression. The view name is represented by v.

Once a view is defined, the view name can be used to refer to the virtual relation that the view generates.

VIEWS

A view is just a relation, but we store a definition, rather than a set of tuples.

```
CREATE VIEW YoungActiveStudents (name, grade)
AS SELECT S.name, E.grade
FROM Students S, Enrolled E
WHERE S.sid = E.sid and S.age<21
```

Views can be dropped using the DROP VIEW command.

How to handle DROP TABLE if there's a view on the table?

- DROP TABLE command has options to let the user specify this.

Operations on Views

Querying is allowed

Update operations are usually restricted because

- updates on a view may modify many base tables
- there may not be a unique way of updating the base tables

to reflect the update on view

  - view may contain some aggregate values
- Ambiguity where primary key of a base table is not included in view definition.

Restrictions on Updating Views

- Updates on views defined on joining of more than one table are not allowed
- For example, updates on the following view are not allowed
- create a view Professor_Dept with professor ID, department Name and department phone

create view profDept(profId, DeptName, DPhone)
as(select f.empId, d.name, d.phone from professor f, department d
where f.depNo= d.depId);

- Updates on views defined with ‘group by’ clause and aggregate functions is not permitted, as a tuple in view will not have a corresponding tuple in base relation.

Restrictions on Updating Views

Updates on views which do not include Primary Key of base table, are also not permitted
For example, updates on the following view are not allowed
Create a view StudentPhone with Student name and phone number

create view StudentPhone(sname,sphone) as(select name, phone from student);

View StudentPhonedoes not include Primary key of thebase table.

Allowed Updates on Views

Updates to views are allowed only if

- defined on single base table
- not defined using ‘group by’ clause and aggregate functions
- include Primary Key of base table

Views and Security

Views can be used to present necessary information (or a summary), while hiding details in underlying relation(s).

Given YoungStudents, but not Students or Enrolled, we can find students s who have enrolled, but not the cid's of the courses they are enrolled in.

Example Queries

A view consisting of branches and their customers

create view all_customer as (select branch_name, customer_name
from depositor, account where depositor.account_number =
account.account_number)
union
(select branch_name, customer_name
from borrower, loan
where borrower.loan_number = loan.loan_number)

Find all customers of the Perryridge branch

select customer_name
from all_customer
where branch_name = 'Perryridge'

Uses of Views

Hiding some information from some users
Consider a user who needs to know a customer's name, loan number and branch name, but has no need to see the loan amount.

Define a view

(create view cust_loan_data as
select customer_name, borrower.loan_number, branch_name
from borrower, loan
where borrower.loan_number = loan.loan_number)

Grant the user permission to read cust_loan_data, but not borrower or loan

Predefined queries to make writing of other queries easier
Common example: Aggregate queries used for statistical analysis of data

Processing of Views

When a view is created

- the query expression is stored in the database along with the view name
- the expression is substituted into any query using the view

Views definitions containing views

- One view may be used in the expression defining another view
- A view relation v1 is said to depend directly on a view relation v2 if v2 is used in the expression defining v1
- A view relation v1 is said to depend on view relation v2 if either v1 depends directly to v2 or there is a path of dependencies from v1 to v2
- A view relation v is said to be recursive if it depends on itself.

View Expansion

A way to define the meaning of views defined in terms of other views.
Let view v1 be defined by an expression e1 that may itself contain uses of view relations.
View expansion of an expression repeats the following replacement step:

> *repeat*
> *Find any view relation vi in e1*
> *Replace the view relation vi by the expression defining vi until no more view relations are present in e1*

As long as the view definitions are not recursive, this loop will terminate

With Clause

The with clause provides a way of defining a temporary view whose definition is available only to the query in which the with clause occurs.

Find all accounts with the maximum balance

```
with max_balance (value) as
    select max (balance)
    from account
select account_number
from account, max_balance
where account.balance =
max_balance.value
```

Complex Queries using With Clause

Find all branches where the total account deposit is greater than the average of the total account deposits at all branches.

```
with branch_total (branch_name, value) as
    select branch_name, sum (balance)
    from account
    group by branch_name
with branch_total_avg (value) as
    select avg (value)
    from branch_total
select branch_name
from branch_total, branch_total_avg
where branch_total.value >= branch_total_avg.value
```

Note: the exact syntax supported by your database may vary slightly.

– E.g. Oracle syntax is of the form
with branch_total as (select ..),
branch_total_avg as (select ..)
select …

Update of a View

Create a view of all loan data in the loan relation, hiding the amount attribute

```
create view loan_branch as
select loan_number, branch_name
from loan
```

Add a new tuple to loan_branch

```
insert into loan_branch
values ('L-37‘, 'Perryridge‘)
```

This insertion must be represented by the insertion of the tuple

```
('L-37', 'Perryridge', null )
```

into the loan relation